School of Computer Science
FACULTY OF ENGINEERING AND PHYSICAL SCIENCES



UNIVERSITY OF LEEDS

Final Report

Investigating the Impact of Spatial Localisation for Real-time Multi-Style Non-Photorealistic Rendering

Rory Edmonds

Submitted in accordance with the requirements for the degree of Computer Science (High-Performance Graphics and Games Engineering) MEng, BSc

2025/26

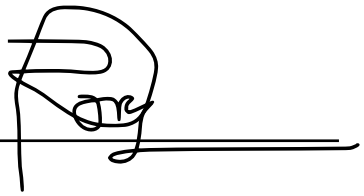
COMP3931 Individual Project

The candidate confirms that the following have been submitted.

Items	Format	Recipient(s) and Date
Final Report	PDF file	Uploaded to Minerva (06/05/26)
Link to online code repository	URL	Sent to supervisor and assessor (06/05/26)

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of Student) 

© 2025/26 The University of Leeds and Rory Edmonds

Summary

This project aims to address the performance and scalability challenges associated with applying multiple non-photorealistic rendering (NPR) effects concurrently in real-time systems. Many image-space stylisation techniques operate as fullscreen post-processing passes, meaning that every pixel on the screen is evaluated regardless of whether stylisation is actually required in that region. Fullscreen stylisation becomes increasingly inefficient as additional effects are introduced, since rendering cost tends to scale with both screen resolution and the number of active styles. This significantly limits the application of many effects in real-time scenarios, such as video games, where rendering must be completed within strict frame-time budgets to maintain interactivity and visual smoothness.

The project investigates whether spatial localisation can reduce redundant computation by restricting stylisation work to only the regions of the screen containing stylised objects. Several localisation strategies are explored, including bounding-box based approaches on both the CPU and GPU, and tile-based approaches, with comparisons made against a traditional fullscreen baseline. The project also examines how factors such as object count, screen coverage, overlap, occlusion, and style combinations influence performance and scalability. By adapting ideas from segmentation and compositing workflows into a real-time context, it seeks to determine under what conditions spatial localisation provides meaningful performance benefits while still maintaining visually correct multi-style rendering.

Acknowledgements

To Mum, Dad and Este (I guess...), I greatly appreciate all your the love and support that has got me this far; I hope I did you proud. My beautiful pets Liquorice, Treacle and Rio, whilst useless for advice, were incredibly helpful in being adorable and cuddly and only sometimes waking me up in the middle of the night with half-dead rodents.

I must of course thank my amazing group of friends who have been absolutely irreplacable, knowing that I am not alone in this has meant more than anything ESPECIALLY when I'm delirious at 4am in Bragg. In particular, I would like to thank Evan Madurai, Miranda Gough, Paige Uppal, Yas Bal, Eschal Najmi, and "george hester".

I would also like to extend thanks to Markus Billeter and Ingrid Morgan for providing much needed help in giving this project direction and showing me lots of interesting graphics projects!

Arguably the most crucial to my dissertation were peppermint tea bags, Lebanese Bites, Taste the Orient and Vertical Fitness Society, as they kept me fed, hydrated and occasionally upside down.

Contents

1 Introduction and Background Research **1**

1.1 Introduction . . . 1

1.1.1 Project Objectives . . . 1

1.2 Background Research . . . 2

1.2.1 Multi-Pass Rendering . . . 3

1.2.2 Image Space Stylisation . . . 3

1.2.3 Segmented Stylisation . . . 4

1.2.4 Compositing Workflows . . . 5

1.2.5 Real-time Rendering . . . 6

2 Methodology **7**

2.1 Project Management . . . 7

2.1.1 Design Constraints . . . 8

2.2 Framework Selection . . . 8

2.2.1 Rendering Engine . . . 8

2.2.2 Rendering Pipeline . . . 8

2.3 Design of the Solution . . . 8

2.3.1 Tagging System . . . 9

2.3.2 Custom Pass Structure . . . 9

2.4 ID Prepass . . . 9

2.5 Spatial Localisation . . . 10

2.5.1 Bounding Boxes . . . 10

2.5.2 Tiling Methods . . . 13

2.6 Image Effects . . . 13

2.7 Evaluation Methodology . . . 13

2.7.1 Measurements Taken . . . 14

3 Implementation and Validation **15**

3.1 System Overview . . . 15

3.2 RenderGraph Execution Model . . . 15

3.3 Scriptable Renderer Feature . . . 15

3.3.1 NprConfig . . . 15

3.3.2 Stylised Tag System . . . 16

3.4 Frame Data . . . 16

3.5 Custom Passes . . . 17

3.6 ID Prepass . . . 17

3.7 Bounding Box Prepasses . . . 18

3.7.1 Compaction . . . 19

3.8 Tiling Prepass . . . 19

3.9 Source Prepass . . . 20
3.10 Effect Passes . . . 20
3.11 Validation . . . 21

4 Results, Evaluation and Discussion 23
4.1 Experimental Setup . . . 23
4.2 Results . . . 23
4.2.1 Number of Styles . . . 23
4.2.2 Screen Coverage . . . 23
4.2.3 Effect of increasing object count . . . 24
4.2.4 Overlap Tests . . . 25
4.2.5 Occlusion Test . . . 26
4.2.6 Effect of Tile Size . . . 27
4.3 Conclusions . . . 28
4.4 Future Work . . . 29
4.4.1 Effects Extending Beyond the ID Buffer . . . 29
4.4.2 Adaptive Tiling . . . 30

References 31

Appendices 34

A Self-appraisal 34
A.1 Critical self-evaluation . . . 34
A.2 Personal reflection and lessons learned . . . 35
A.3 Legal, social, ethical and professional issues . . . 35
A.3.1 Legal issues . . . 35
A.3.2 Social issues . . . 36
A.3.3 Ethical issues . . . 36
A.3.4 Professional issues . . . 36

B External Material 37

C Specs 38

D Full Graphs 39

Chapter 1

Introduction and Background Research

1.1 Introduction

The field of Non-Photorealistic Rendering (NPR) consists of rendering techniques designed to produce stylised visuals rather than physically based images. It has many applications in interactive real-time environments, such as video games, where visual identity and clarity are important design considerations.

Applying multiple NPR effects concurrently introduces significant computational overhead as many NPR techniques operate uniformly across the entire screen. While this is appropriate for globally stylised scenes it becomes inefficient in cases where stylisation is only required for a subset of objects or regions. This introduces a scalability issue; with each additional effect applied, rendering cost tends to grow proportionally with screen resolution. This limits the viability of multi-style rendering in real-time systems as they are constrained by tight frame-time budgets and as such require high-speed computation. The utility of rendering multiple distinct art styles in a scene is seen in animated film production pipelines, like *Spiderman: Into the Spiderverse* [33], where different visual effects are applied selectively and combined in post-processing. These pipelines rely on offline rendering to achieve this and aren't directly suitable for real-time application.

This project investigates whether similar principles can be adapted to real-time rendering by introducing spatial localisation of stylisation. Instead of applying effects across the full screen rendering work will instead be restricted to regions where stylised objects are present.

The driving hypothesis is that by localising where stylisation occurs then redundant computation, caused by evaluating pixels that don't require any effects to be applied, is reduced and prevents rendering cost from scaling as aggressively with the number of styles. This is evaluated through a series of controlled experiments measuring the performance of the system under varying scene complexity, screen coverage and style combinations.

1.1.1 Project Objectives

The primary objective of this project is to investigate whether spatial localisation can improve the scalability of multi-style NPR in real-time systems. To achieve this, the project aims to:

- Design and implement a pipeline that supports multiple concurrent stylisation effects in *Unity* [42].
- Develop methods for localising rendering work.
- Evaluate how rendering cost scales with styles applied, coverage of stylised regions, object count, overlap, occlusion and object granularity.
- Analyse whether localisation reduces redundant computation compared to full-screen rendering approaches.

1.2 Background Research

NPR refers to a class of rendering techniques that prioritise stylisation over physical accuracy. Rather than simulating the behaviour of light to produce realistic images, as found in photorealistic rendering, NPR methods aim to generate imagery that conveys a particular artistic or illustrative style. As described by Kyprianidis et al. [24], such methods are often inspired by art forms and aim to replicate them whilst taking advantage of the computational power of modern rendering pipelines.
Painterly rendering techniques involve reconstructing images using discrete, form-aligned brush strokes as shown in Figure 1.1b [18, 17, 26]. Other notable approaches simulate physical processes such as the pigment diffusion found when using watercolour paints on paper [7]. Stroke-based techniques have also been adapted to emulate the hatching and stippling techniques used in pen-and-ink illustrations as shown in Figure 1.1a [18, 20]. However, these methods often struggle with temporal coherence in animated contexts, leading to artefacts such as the *shower door* effect, where strokes appear fixed relative to the screen rather than the underlying geometry. Beyond these artistic representations, NPR also covers scientific and



(a) Pen-and-ink sketch style renders [18]



(b) Painterly stylisation applied to a photograph [18]

Figure 1.1: Stroke-based NPR techniques

technical illustration where the stylisation serves a functional rather than aesthetic purpose. By abstracting away unnecessary details, the clarity of such illustrations is improved.Figure 1.2 illustrates the solutions proposed by Gooch et al. that modify lighting models in order to highlight the structure of 3D Geometry [14] .

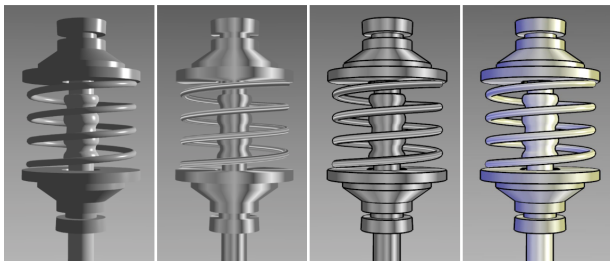


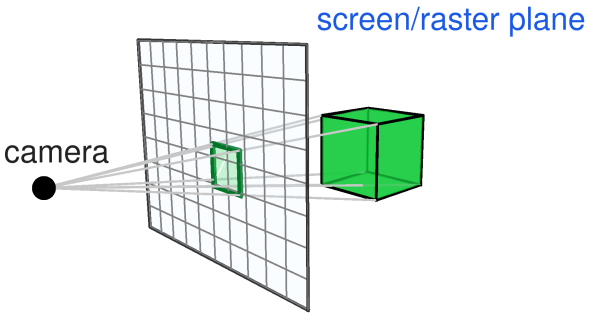
Figure 1.2: Visual comparison of different lighting models [14]

Additional techniques found in NPR projects aim to recreate the constraints imposed on digital artists in the past. Methods such as dithering were initially used in systems with limited colour palettes to represent tonal variations through patterns that the human eye averages out to smooth gradients [38]. While current hardware is no longer bound by these restrictions, usage of this technique is found in games such as *Return of the Obra-Dinn* [28] to emulate this era. The same philosophy is present with effects like toon shading; this quantises colour into

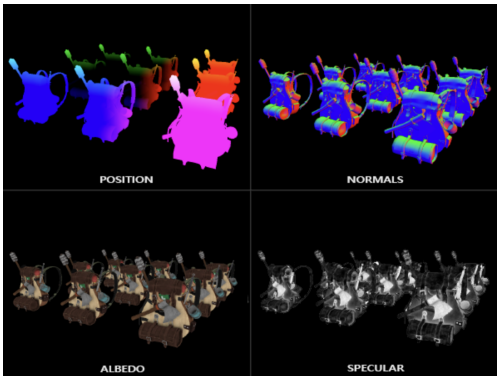
discrete bands in order to reproduce the comic-book style of drawing [4, 36, 15]. A notable usage of this on modern hardware is *Legend of Zelda: Breath of the Wild* [30] which received critical acclaim for its visual style [19].
While these techniques demonstrate the breadth of NPR, they are typically studied in isolation. The challenge of combining multiple styles within a single real-time system introduces additional complexity, particularly in terms of performance and compositing, which is less explored in existing literature.

1.2.1 Multi-Pass Rendering

The process of 3D rendering involves transforming the input scene, defined by geometric objects, materials, and lighting, into a 2D image displayed on the screen through a sequence of stages in the graphics pipeline. Scene geometry is processed and projected from 3D world space into screen-space through rasterisation, as shown in Figure 1.3a. This results in a grid of pixels that are each then shaded, meaning its resulting colour is computed based on its lighting and material properties. Modern rendering systems break this process down into multiple passes. A render pass performs a specific operation on the scene and produces intermediate outputs, with Figure 1.3b showing examples. These are stored in buffers and can be sampled by following passes, which allows complex visual effects to be constructed incrementally.



(a) Projection of 3D geometry onto the screen-space grid



(b) Example intermediate buffers produced during a multi-pass pipeline [9]

1.2.2 Image Space Stylisation

Image space techniques operate on the intermediate screen-space buffers produced by the rendering pipeline, such as: colour, depth, and normals. Common examples include edge detection filters, which identify discontinuities in depth or surface orientation [2]. Image-space stylisation filters such as eXtended Difference of Gaussians (XDoG), shown in Figure 1.4, enhance edges by subtracting Gaussian-blurred versions of the image at different scales and applying a thresholding step to produce stylised line drawings [45].
By decoupling stylisation from scene geometry, image-space methods enable a more modular and flexible rendering pipeline. Effects can be applied, reordered, or combined independently of the underlying scene representation. This also allows them to be applied selectively over regions of the screen, enabling optimisations based on screen-space coverage.

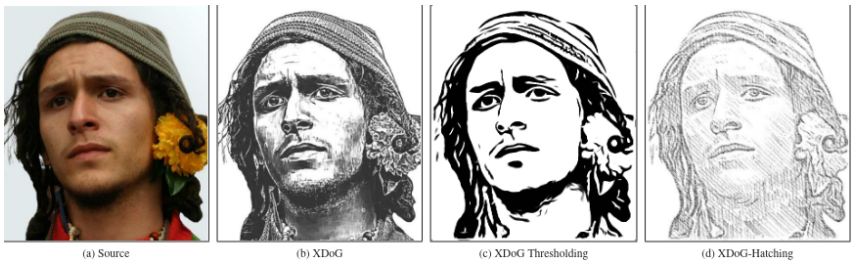


Figure 1.4: XDoG-based image-space stylisation [45]

As such, image-space techniques provide a suitable foundation for investigating the composition and performance of multiple NPR styles in real-time systems. However, this flexibility comes at the cost of reduced access to object level information which can lead to artefacts like the previously mentioned *shower door* effect and challenges when combining multiple effects, particularly in the presence of occlusion and overlapping stylisation.

1.2.3 Segmented Stylisation

To achieve multiple distinct styles in a scene, an NPR framework must allow different regions of an image or object to exhibit distinct visual characteristics. This enables both artistic flexibility and functional benefits, such as emphasising important features. One approach to achieving this is through the use of control maps or weighting buffers, which modulate the influence of stylisation across a surface or image. The MNPR pipeline, for the 3D Modelling software *Maya* [3], used these methods to vary the intensity or type of stylisation applied at different locations, enabling detailed artistic control while maintaining coherence across the scene [29]. The use of such techniques to partition the image into stylistic regions has been shown to improve scene understanding and guide viewer attention, as different stylisation cues can be used to highlight important elements [34, 23]. Wegen et al. [12] provide an example in the context of rendering point clouds, where stylisation is used to clarify the noisy data associated with the format. They also introduce a depth-aware blending strategy to address compositing artefacts that arise when multiple stylised regions overlap. Their work highlights that, while segmented stylisation can improve clarity, correctly combining stylised outputs in screen-space remains a challenge, particularly in the presence of occlusion.

A real-time segmentation solution by Bleron [6] encountered the problem that stylisations produced by their approach were unable to extend past the G-Buffer, a set of screen-space buffers storing per-pixel geometry information. A naive inflation of the G-Buffer to account for such effects resulted in overlapping artefacts with other objects as shared pixels would only represent that most recent fragment written during rasterisation; shown in Figure 1.5. To overcome this issue they proposed a local segmentation strategy that identifies multiple overlapping surface contributions within a filter neighbourhood. By clustering per-pixel data into distinct surface components, stylisation operations such as inflation and filtering can be applied independently to each surface part before being recomposited into a final image. This improves visual coherence in overlapping regions but does introduce additional computational overhead, as the operation must be carried out for each neighbourhood present on the screen.

A more lightweight approach to segmentation is seen from Spindler et al. who use additional

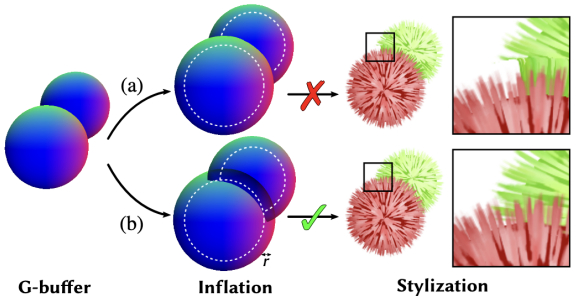


Figure 1.5: Naive G-buffer inflation versus segmentation-based stylisation [6]

buffers containing object identifiers and foreground masks to separate different regions of the scene [36]. This allows stylisation effects to be applied selectively but doesn't support the extension of effects past this predefined buffer. However, its simplicity is well suited to real-time applications which have strict computational restrictions.

1.2.4 Compositing Workflows

While intermediate buffers store different aspects of the scene, compositing defines how these outputs are combined to form the final image. Rather than applying all effects in a fixed order, compositing allows the outputs of different render passes to be selectively blended, masked, or layered [5].

However, many conventional render passes store only a single visible surface contribution per pixel which limits their ability to represent scenes with overlapping geometry or partial transparency. The Cryptomatte [13] data format encodes multiple object IDs and pixel coverages for each. This multilayer representation allows compositing operations to distinguish between overlapping contributions within a single pixel and reliably extract object regions without manual masking - an example is in Figure 1.6. Developed for *The Wild Robot*, Crypto-Deep [44] extends this by storing additional per-layer attributes such as surface position and normals, providing a more expressive representation while avoiding the full memory cost of traditional deep image formats. As noted by Verlander et al. [44], traditional deep data can be "10–40 times larger in file size causing a higher memory requirement for compositing software and a slow-down in interactive working speed". By reducing this overhead, Crypto-Deep enables more practical compositing of partially transparent elements, allowing effects such as painterly foliage to be achieved while maintaining interactivity for the artists.



Figure 1.6: Image of rendered scene and associated Cryptomatte buffer [37]

In Spiderman: Into the Spiderverse the *Patcher Tool* [35] was used heavily to add painterly effects onto the 3D renders in post-processing. Whilst Cryptomatte and Crypto-Deep accurately represent scene elements, this tool partitions the rendered image into regions, known as *patches*, derived from a position pass so stylisation remains coherent to the underlying

geometry. These regions then have their colour averaged to reduce local detail while preserving the structure of the image. Finally, a distortion step is applied to introduce irregularity in the patches to mimic handpainted variation. The *Patcher Tool* was extended in the sequel *Spiderman: Across the Spiderverse* [11] by introducing scanned brushstroke textures and additional brush based tools, including projected brush patches, 3D cards, and 3D curves [8]. Rather than relying solely on lower resolution colour patches, the pipeline now allows brush shapes to be distributed over scene geometry. Through the randomisation of brushstroke characteristics and by compositing multiple stroke layers together, the system better simulated the accumulation and irregularity of handpainted media.

1.2.5 Real-time Rendering

When working within a real-time rendering system we must impose much stricter constraints on the performance of multiple style compositing. Production pipelines are designed for rapid artistic iteration as opposed to frametime budgets so although they are interactive they often rely on approximations to achieve this during production.

For example, Verlander et al. [44] only evaluates a subset of the Cryptodeep layers during interactive workflow, with full processing deferred to offline rendering. This reflects a broader distinction between production and real-time contexts: while production pipelines can distribute computational cost over a large timespan, real-time systems must evaluate all stylisation effects within a tight time budget each frame to ensure a smooth experience for users [1]. This greatly restricts both the number of rendering passes and the amount of per-pixel computation that can be performed.

From a GPU perspective fragment processing cost scales with the number of rasterised pixels. As described by Akenine-Möller et al. [1], many real-time rendering techniques aim to minimise unnecessary fragment shading by restricting computation to spatially relevant regions. For example, deferred shading pipelines commonly employ light volumes (e.g. spheres or cones) to ensure that shading is only performed for pixels within a light's area of influence [16]. At a lower level, graphics APIs such as *OpenGL* [22] provide mechanisms such as scissor testing to restrict rendering to rectangular regions of the screen, avoiding redundant rasterisation [21]. Tile-based shading partitions the screen into a regular grid of fixed-size regions and assigns lights to each tile, allowing each pixel to evaluate only the lights affecting its region [31]. Olsson et al. [32] extend this idea by subdividing the view frustum into 3D clusters, improving light assignment in scenes with significant depth variation. Together, these approaches demonstrate that spatial partitioning can reduce unnecessary lighting work while remaining well suited to GPU parallelism. Despite these techniques, similar spatial restriction strategies have not been applied to image-space stylisation pipelines combining multiple concurrent effects, which is the gap this project seeks to address.

Chapter 2

Methodology

2.1 Project Management

Development was managed using an iterative, sprint-based approach inspired by the Scrum framework. This methodology was selected because the project direction was not fully determined at the outset, with the primary research contribution of the performance impact of spatial localisation emerging through experimentation rather than being fixed in advance. Short iteration cycles allowed hypotheses about performance and correctness to be tested incrementally, with each sprint producing a working, testable build rather than deferring integration to the end of the project. The full sprint structure is described in Table 2.1.

Sprint	**Goal**	**Key Deliverables**
1	Research and familiarisation	NPR literature review; Unity rendering architecture analysis
2	Renderer feature and pipeline foundation	Renderer feature; test pass to validate function
3	Core Pipeline	ID prepass generating per-pixel style masks; per-object bitmask tagging system
4	Fullscreen Baseline Completed	2 simple NPR effects added for correctness testing; dummy test effect implemented for controlled benchmarking
5	CPU bounding-box localisation	CPU bounding-box generation, initial benchmarks against fullscreen
6	GPU bounding-box localisation	GPU compute bounding-box generation; batched instanced draws; additional benchmarks
7	Occlusion culling	Occlusion pass; visibility buffer; benchmarks under controlled occlusion
8	CPU merging and GPU merging attempt	CPU merge pass; GPU merging explored but found architecturally unsuitable
9	GPU tile-based compaction	Tile rasterisation kernel; tile region emission; ablation benchmarks
10	GPU tile-based generation and evaluation	Fully screen-space tile pipeline; final benchmarking across all strategies
11	Report consolidation	Results analysis; report writing; evaluation and self-appraisal

Table 2.1: Sprint plan with key deliverables

Version control was managed throughout using Git, with the repository hosted on GitHub. Each task or sub-task was developed on a dedicated branch and merged into `main` via a pull request upon completion. This ensured that the `main` branch always represented a stable, working state of the system. A Unity-specific `.gitignore` was maintained to exclude generated asset metadata and build artefacts from the repository.

2.1.1 Design Constraints

The following constraints are derived from the project objectives established in subsection 1.1.1. The pipeline must support multiple concurrent NPR effects assignable at the object level, with rendering cost scaling with stylised region coverage rather than total screen resolution. It must maintain real-time frame rates with consistent delivery, remain visually correct under occlusion and overlap, and be structured such that individual localisation strategies can be substituted independently to support controlled benchmarking.

2.2 Framework Selection

This section details the software frameworks utilised in this project and the reasoning behind those decisions

2.2.1 Rendering Engine

Unity was selected as the rendering environment for this project. As a widely adopted engine with extensive documentation and an active developer community, Unity provides a stable foundation for custom rendering research without the overhead of constructing a rendering pipeline from scratch. Low-level graphics APIs such as OpenGL or Vulkan would offer greater control but would require a complete rendering pipeline to be implemented before any NPR work could begin, diverting effort from the spatial localisation investigation itself.

2.2.2 Rendering Pipeline

Unity exposes two rendering pipeline frameworks: the Built-in Pipeline and the Scriptable Render Pipeline (SRP). The Built-in Pipeline does not support custom pass insertion at arbitrary stages, making it unsuitable for the multi-pass compositing architecture required by this project. The SRP was therefore adopted, as it allows custom passes to be explicitly scheduled within the pipeline. Within the SRP, Unity provides the Universal Render Pipeline (URP) and the High Definition Render Pipeline (HDRP) as reference implementations to build from. URP was selected over HDRP as it provides a lightweight, modern pipeline appropriate for the scale of this project. HDRP introduces substantial additional complexity oriented towards photorealistic rendering, which provides no benefit for an NPR investigation. A `ScriptableRendererFeature` is Unity's mechanism for extending the URP pipeline with custom rendering logic. It provides a structured entry point for creating and scheduling `ScriptableRenderPass` instances, which are then executed by the renderer in a defined order. This makes it the natural choice as the central controller for the NPR pipeline, as it allows the full set of custom passes to be treated as a single extension to the existing renderer rather than a collection of independent modifications.

2.3 Design of the Solution

The system has two main components: the per-object tagging system that will allow the renderer to know what styles to assign to what objects, and the pipeline itself that will perform localisation and apply the desired effects to the output image.

2.3.1 Tagging System

To achieve per-object stylisation, each object will be assigned a style bitmask in which each bit corresponds to a specific image-space effect. This representation supports the assignment of multiple concurrent effects to a single object whilst remaining efficient to evaluate through standard bitwise operations. This design decouples stylisation intent from rendering implementation with objects being configured independently of the pipeline. The pipeline reads style information from the ID texture at the shader level rather than querying scene objects directly, ensuring that styles can be modified at the scene level without requiring changes to the rendering pipeline itself.

2.3.2 Custom Pass Structure

The pipeline consists of three sequential stages, illustrated in Figure 2.1. First, an ID prepass renders each stylised object into a screen-space texture; giving a pixel-precise map of where effects should be applied. The localisation stage then the screen-space regions requiring stylisation for each effect, either from the world-space bounds of tagged objects or directly from the identity texture depending on the active strategy. Finally, each image-space effect is applied over only these regions, restricting fragment processing to where stylised objects are present rather than the full screen.

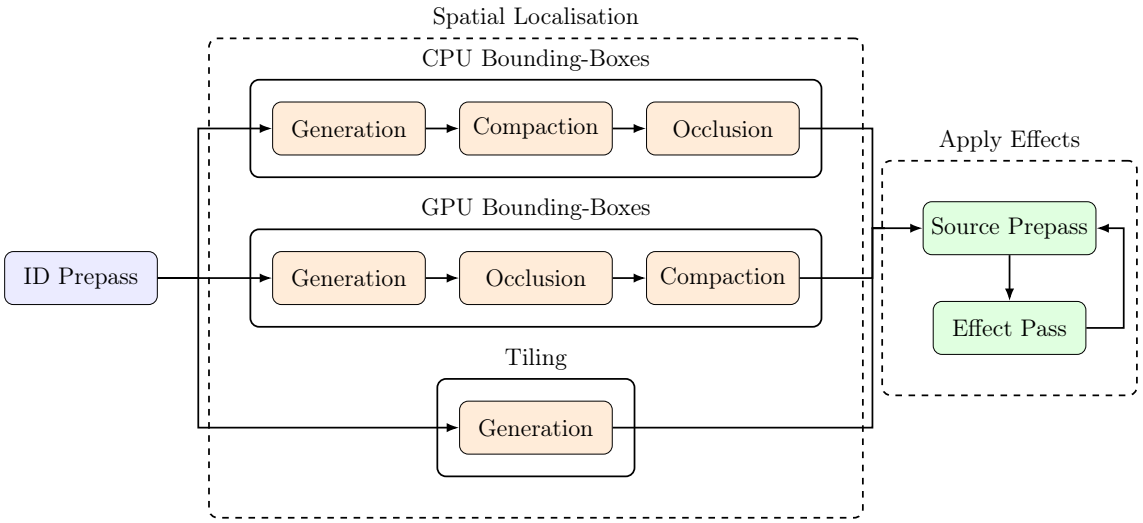


Figure 2.1: Pass ordering in NPR pipeline

2.4 ID Prepass

Each stylised object is encoded into a screen-space identity texture using a per-style bitmask, where each bit corresponds to a single active effect. This produces a pixel-accurate map of style membership across the frame, to ensure effect application is restricted to relevant pixels regardless of the active strategy. Encoding by style rather than by object allows multiple objects sharing a style to be processed together in a single batched pass, scaling with the number of active styles rather than the number of objects in the scene.

2.5 Spatial Localisation

This section describes the three main approaches used to localise NPR effect execution: CPU & GPU bounding-box based localisation techniques, and tile-based localisation.

2.5.1 Bounding Boxes

Bounding boxes were the initial localisation strategy, motivated by the established use of bounding volumes in restricting processing to spatially relevant regions, as seen in deferred shading light volumes discussed in subsection 1.2.5. They are constructed and manipulated over several render passes that populate intermediate buffers with the region restricting information to be used in optimising future effect passes. Each stage of the bounding-box pipeline is implemented in both a CPU-sequential and a GPU-parallel variant, enabling direct performance comparison of each approach under controlled conditions.

Generation

Each bounding-box is defined by its centre $\mathbf{c}$ and extents $\mathbf{e}$ in world space. The goal is to project these bounds into screen-space to produce rectangles that conservatively approximate the regions requiring processing. Naïvely projecting all eight corners produces erratic regions when any corner lies behind the near-plane of the camera. To address this, each bounding-box is decomposed into its 12 edges, represented as parametric line segments $\mathbf{p}(t) = \mathbf{a} + t(\mathbf{b} - \mathbf{a})$ for $t \in [0, 1]$, where $\mathbf{a}$ and $\mathbf{b}$ are the edge endpoints in camera space. These are then clipped against the near-plane to ensure a valid projection occurs. In a right-handed camera space, visible points satisfy $z \leq -z_{\text{near}}$. For edges that cross the near plane, the intersection parameter is found by solving $a_z + t(b_z - a_z) = -z_{\text{near}}$, giving:

$$t^* = \frac{-z_{\text{near}} - a_z}{b_z - a_z}.$$

Vertices satisfying $z \leq -z_{\text{near}}$ are retained, intersection points at t^* are added for clipped edges, and vertices behind the near plane are discarded, as illustrated in Figure 2.2. The retained vertices are projected into screen-space, and the final bounding rectangle is taken as the axis-aligned bounding-box of the projected point set, clamped to the viewport extents.

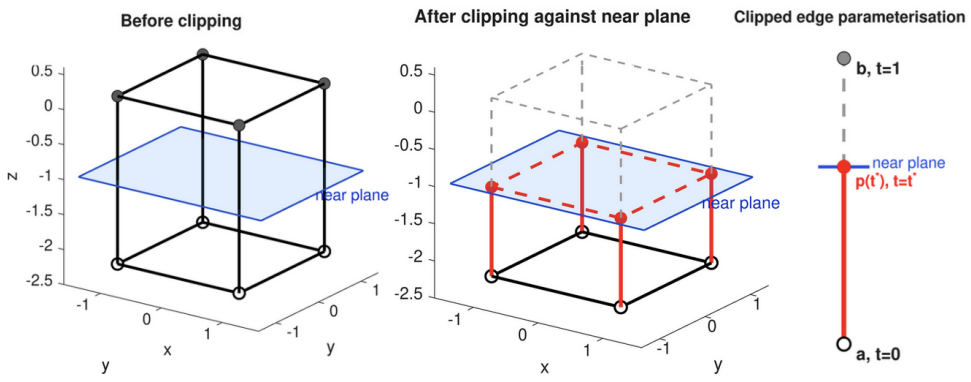


Figure 2.2: Near-plane clipping of bounding-box edges prior to screen-space projection

Occlusion Pass

Frustum culling, which discards objects outside the camera's view volume, does not account for occlusion by foreground geometry. As a result, bounding-boxes may be produced for objects that are fully hidden from the camera, leading to unnecessary processing of a bounding-box that contains no pixels to stylise. To mitigate this, the ID texture is sampled within each bounding-box region. If no pixel contains the corresponding style bitmask, the bounding-box is flagged and excluded from effect application as shown in Figure 2.3. A tradeoff that this approach brings is that if a foreground object sharing the same style overlaps the bounding-box of an occluded object, the occluded object will be incorrectly flagged as visible. This is a conservative consequence and does not justify using a per-object identifier over the more scalable per-effect identifier.

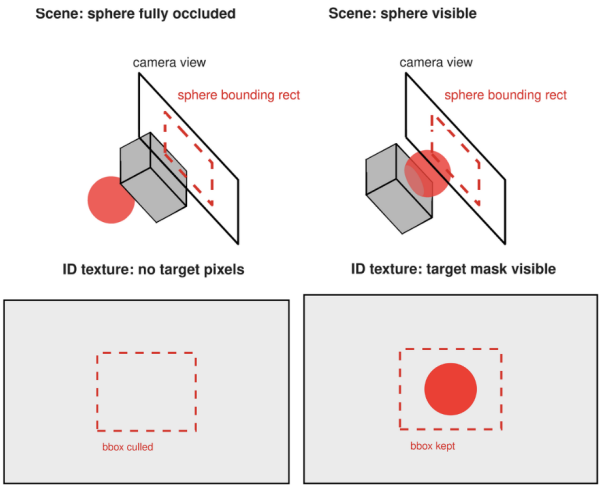


Figure 2.3: Occlusion testing using the ID texture

CPU Compaction Pass

When multiple stylised objects with overlapping screen-space extents share a style, their bounding-boxes may cause the same pixels to be processed by multiple draw calls. An iterative pairwise merging algorithm is applied to reduce this, illustrated in Figure 2.4. Let R_i and R_j be axis-aligned bounding-boxes with style masks m_i and m_j. A merge is considered only when $m_i \,\&\, m_j \neq 0$, as merging disjoint-style boxes would introduce unnecessary fragment processing rather than reduce it. When this holds, a merge is performed only if the area of overlap between the two boxes exceeds the empty area that would be newly enclosed by their union:

$$A(R_i \cap R_j) > A(\text{bbox}(R_i \cup R_j)) - A(R_i) - A(R_j).$$

The merged rectangle is assigned the shared mask $m_{ij} = m_i \,\&\, m_j$, with remaining bits retained in the originals. This repeats until no further beneficial merges are possible. An iterative pairwise approach is used over optimal merging over all subsets as real-time frame budgets require speed over precision. The area condition ensures each merge strictly reduces total processed area, ensuring the same state cannot be revisited and thus termination is guaranteed.

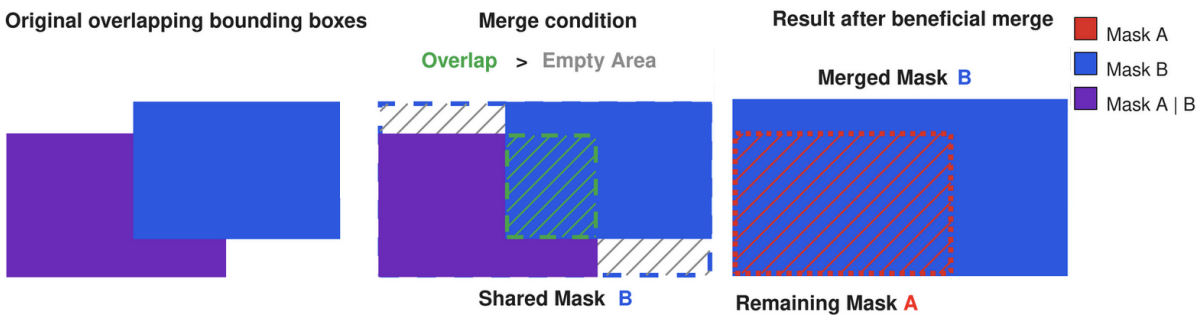


Figure 2.4: Bounding-box compaction steps

Tile-Based Compaction Pass

The iterative nature of the CPU compaction algorithm is not suited to batch execution on the GPU, as each merge depends on the result of the previous and requires global knowledge of the bounding-box set. The tile-based representations explored in subsection 1.2.5 provide an alternative approach that is better suited to the parallel nature of the GPU. Each visible bounding-box is rasterised into a fixed screen-space tile grid. Let the screen be partitioned into tiles of size $T \times T$ with indices (t_x, t_y). For a bounding-box $R_i = (x_i, y_i, w_i, h_i)$ with style mask m_i, the set of tiles covered by R_i is:

$$t_x \in \left[\left\lfloor \frac{x_i}{T} \right\rfloor, \left\lfloor \frac{x_i + w_i - 1}{T} \right\rfloor \right], \quad t_y \in \left[\left\lfloor \frac{y_i}{T} \right\rfloor, \left\lfloor \frac{y_i + h_i - 1}{T} \right\rfloor \right].$$

Each tile accumulates the style masks of all bounding-boxes that cover it:

$$M(t_x, t_y) = \bigvee_{i \,:\, (t_x, t_y) \in R_i} m_i,$$

Tiles with $M(t_x, t_y) \neq 0$ are emitted as output regions $R_t = (t_x T,\ t_y T,\ w_t,\ h_t)$, clamped to screen boundaries as shown in Figure 2.5. Compaction is achieved implicitly: overlapping bounding-boxes contribute to shared tiles rather than requiring explicit rectangle unions, avoiding iterative algorithms and global synchronisation.

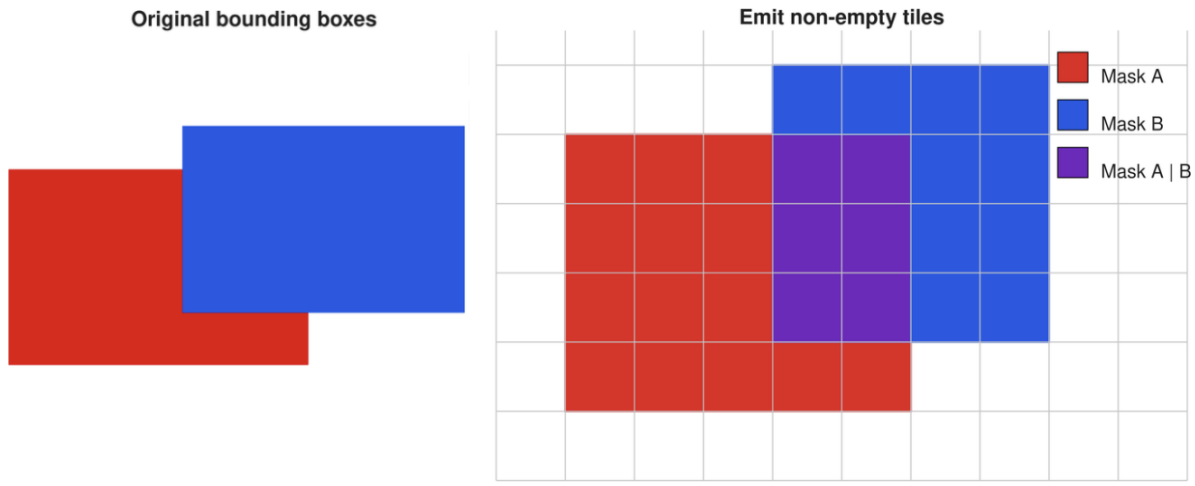


Figure 2.5: Compaction of bounding-boxes using tile rasterisation

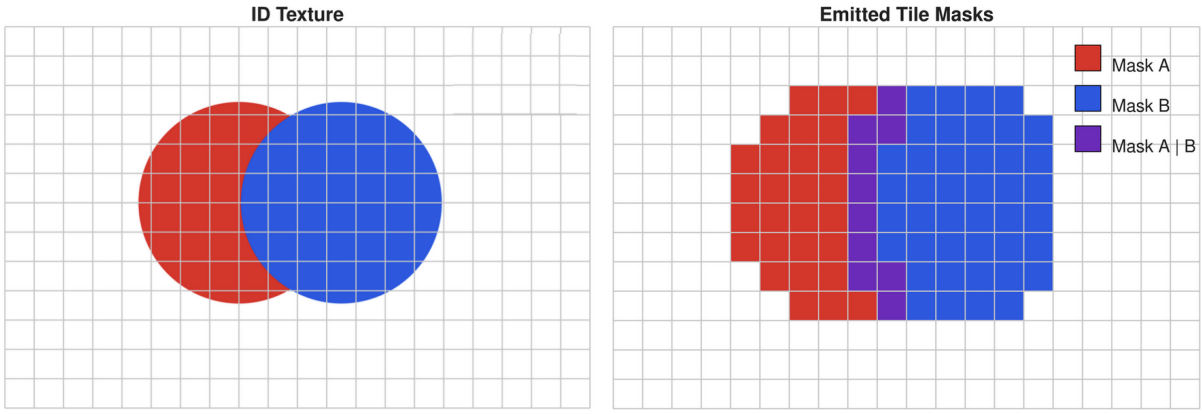


Figure 2.6: Generation of tile regions from the ID texture

2.5.2 Tiling Methods

Tiling is also applied as a standalone localisation strategy, deriving active regions directly from the ID texture rather than from object bounds. Each tile (t_x, t_y) accumulates the style masks of all pixels within its extent:

$$M(t_x, t_y) = \bigvee_{(x,y)\in(t_x,t_y)} m(x, y),$$

Non-empty tiles are emitted as regions, depicted in Figure 2.6. This approach performs localisation entirely in screen-space, removing the need for bounding-box generation, near-plane clipping, or occlusion testing, providing a simpler and fully GPU-driven pipeline.

2.6 Image Effects

Each image-space effect is applied over the screen-space regions produced by the localisation stage or the whole screen in the baseline case. The ID texture is then used to restrict processing to tagged pixels within each region. A source pass captures the current colour buffer immediately before each effect, allowing effects to read from and write to the colour target independently and compose correctly when applied sequentially. Depending on the localisation strategy used, effects are applied either sequentially per region on the CPU, or in parallel by dispatching all regions simultaneously on the GPU, with each region processed as an independent instance.

2.7 Evaluation Methodology

The evaluation aims to determine under what conditions each spatial localisation strategy outperforms naive fullscreen rendering, and how each approach scales as scene complexity increases. The benchmarking methodology is inspired by *A Benchmark for Animated Ray Tracing* (BART), which advocates using parametrically controlled scenes and a consistent measurement protocol to enable fair comparison between rendering algorithms [25]. Following this principle, test scenes are generated programmatically with a single variable changed at a time, keeping all other parameters fixed.

Style count: The number of concurrent NPR effects active in the scene. This measures the impact of multiple stylisation passes.

Object count: The number of stylised objects in the scene, with screen-space coverage held constant. This isolates the overhead associated with managing larger numbers of regions, independent of total stylised area.

Screen-space coverage: The proportion of the screen occupied by stylised objects. This evaluates whether localised rendering cost scales with stylised area rather than total screen resolution.

Occlusion: The proportion of stylised regions hidden by occluding geometry. This evaluates whether the occlusion pass successfully reduces redundant processing for regions containing no visible stylised pixels.

Overlap: The degree to which stylised regions overlap in screen space. This evaluates the effectiveness of compaction strategies under conditions where redundant region processing is most prevalent.

Object size: The size of the objects in the scene. This evaluates the impact of granularity on image localisation techniques.

2.7.1 Measurements Taken

Mean frame time is the primary performance metric, reflecting the per-frame timing constraints of real-time rendering [1]. Standard deviation is reported alongside the mean to characterise typical frame-time spread, giving an indication of consistency under normal conditions. However, standard deviation alone can understate the severity of worst-case frames [10]. Occasional spikes can produce perceptible stuttering even when the mean is acceptable, and are therefore an important aspect of evaluation in their own right [27]. The 95th and 99th percentile frame times are therefore reported alongside the mean as tail metrics, normalised to show:

$$\text{tail overhead} = \frac{p_k - \bar{t}}{\bar{t}}$$

where p_k is the kth percentile frame time and $\bar{t}$ is the mean frame time. A value of zero indicates perfectly consistent frame delivery; larger values indicate increasingly severe spikes relative to the typical case. Together, mean and standard deviation characterise typical behaviour, whilst the normalised tail metrics capture worst-case stability independently of each method's baseline cost.

Chapter 3

Implementation and Validation

3.1 System Overview

The system is implemented as a custom `ScriptableRendererFeature` within Unity's Universal Render Pipeline, with the overall architecture and design rationale established in chapter 2. This chapter documents the implementation of each pipeline stage and the validation strategies applied throughout development.

3.2 RenderGraph Execution Model

Unity's RenderGraph API underpins all pass execution in this pipeline. It operates on a deferred model where passes are recorded into GPU command buffers before any execution occurs. Each pass declares its resource inputs and outputs through RenderGraph handles, enabling automatic dependency tracking and scheduling without manual resource management. Passes are recorded as either raster passes or compute passes. Raster passes write to render targets and cover the ID prepass, source copy, and effect passes. Compute passes perform GPU-side data processing and cover GPU bounding-box generation, occlusion testing, tile compaction, and tile generation. Two constraints follow from the deferred model that shape the pipeline design: GPU results are unavailable to CPU scheduling logic within the same frame, and a pass cannot read from and write to the same render target within a single execution.

3.3 Scriptable Renderer Feature

The `NprStylesRendererFeature` is the central controller of the pipeline, organised around two Unity lifecycle methods. `Create()` instantiates and configures all pass objects according to the current `NprConfig` state. `AddRenderPasses()` is called once per frame and enqueues passes into the URP rendering pipeline as shown in Figure 3.1.
CPU bounding-box generation and merging perform no GPU-side computation, but are implemented as `ScriptableRenderPass` instances so they participate in the same execution ordering as other passes. The first RenderGraph constraint means that in CPU mode merging must be enqueued before occlusion - occluded boxes may therefore be merged with unoccluded boxes, introducing redundant computation. In GPU mode occlusion precedes merging, as the visibility buffer remains on the GPU and can be consumed directly by the tile compaction pass. The second constraint necessitates a dedicated source prepass before each effect pass.

3.3.1 NprConfig

`NprConfig` is the central configuration interface for the renderer feature, exposing the active render mode, tile size, occlusion, merging, and effect type as configurable parameters. Updating these values and calling `Create()` rebuilds the entire pass set to reflect the new configuration. The feature also supports a test mode in which standard effects are replaced

with a configurable number of dummy effects, allowing style count and per-fragment cost to be varied independently to isolate pipeline overhead from effect complexity. During benchmarking, the test runner wrote to `NprConfig` programmatically to cycle through the full matrix of rendering modes and ablation variants automatically between runs, in accordance with the evaluation methodology described in section 2.7.

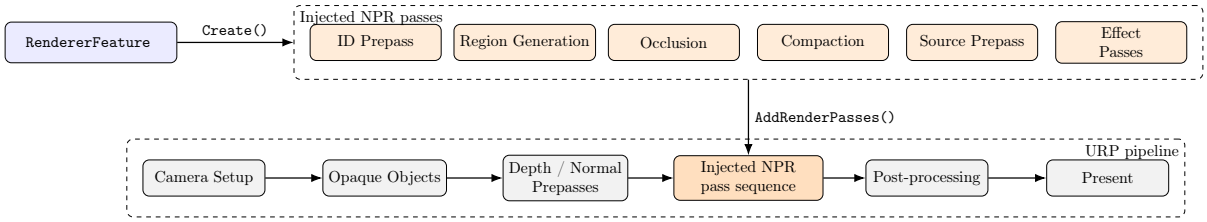


Figure 3.1: Depiction of NPR pass injection into the URP RenderGraph execution sequence

3.3.2 Stylised Tag System

The `StylisedTag` component is the entry point into the pipeline. It stores a style bitmask constructed from the `ImageSpaceEffect` enum or a boolean list of test indices when in testing mode for the object it is attached to. These components are searched for by the bounding-box generation prepasses in order to find stylised objects. When `Apply()` is called, this bitmask is written to the `_ImageStyleID` shader property of all associated renderers which are then read by the ID prepass to generate the necessary texture.

3.4 Frame Data

`NprFrameData` is the shared data container that propagates intermediate results between passes each frame. The ID texture is written to it initially, followed by the tiling or bounding-box generation stages populating it with region data, occlusion passes with visibility data, and compaction passes refining the region data. Before each effect, the source prepass also writes an updated source texture into it. The effect passes then read from this final state of `NprFrameData` to obtain the source and ID textures along with the region data to apply stylisation. Figure 3.2 illustrates how data flows between stages using this structure.

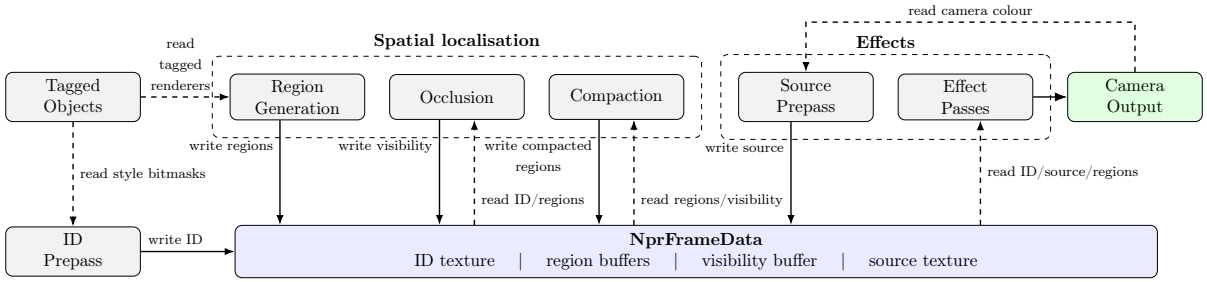


Figure 3.2: Flow of data through `NprFrameData` in the pipeline

Depending on the active rendering mode the region data is either a CPU-side `List<BoundingBox>` or a set of GPU structured buffers: `bboxRectBuffer` storing region dimensions as `float4(minx,miny,width,height)` values, `bboxMaskBuffer` storing per-region

style bitmasks, `bboxCountBuffer` storing the number of active regions, and `bboxIndirectArgsBuffer` storing draw arguments for instanced effect passes. For both bounding-box methods there is also the GPU-side `bboxVisibilityBuffer` storing binary occlusion flags. Buffer indices are shared across all GPU buffers: a given index refers to the same bounding box in every buffer simultaneously, so consistency must be maintained whenever any buffer is written, replaced, or reordered.

3.5 Custom Passes

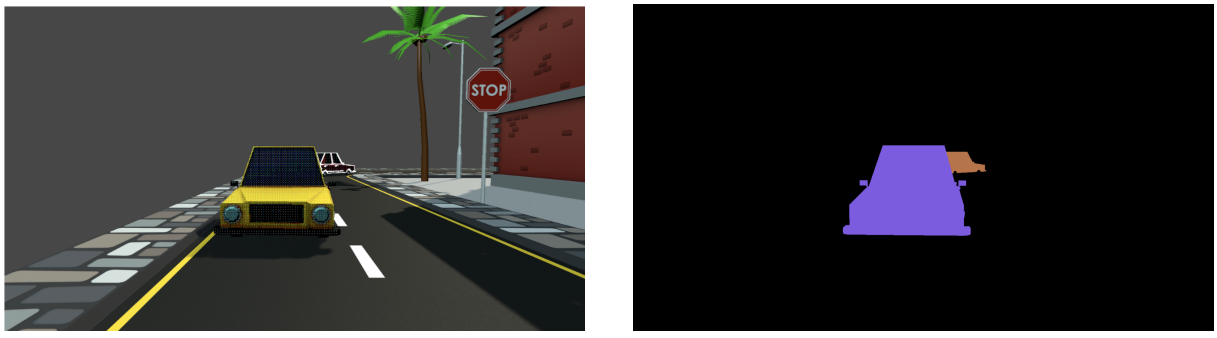
The ScriptableRenderPasses are split into two categories for modularity. Prepasses generate the intermediate data consumed by effect passes, so a common `Prepass` base class was used to enforce consistent pass naming and lifecycle management to them. All passes that draw effects to the camera output inherit from the `EffectPass` base class, which enforces a unified execution structure across rendering modes so benchmark differences are attributable to localisation strategy instead of effect implementation.

3.6 ID Prepass

The ID prepass renders scene geometry into a dedicated screen-space texture, writing the `_ImageStyleID` value, assigned by `Stylised Tag`, of each stylised renderer via an override shader. The override shader replaces all materials on the objects with the `_ImageStyleID` value. The render target uses an `R8G8B8A8_UNorm` format, encoding the full 32-bit style mask across four 8-bit channels [r,g,b,a]. The bitmask can then be reconstructed by shifting each channel accordingly during sampling in effect shaders.

```
uint pixelMask = r | (g << 8) | (b << 16) | (a << 24);
```

The pass iterates over all `StylisedTag` components to accumulate the global style mask. This mask is stored in `NprFrameData` and used to gate effect pass recording, preventing passes from being queued when their associated style has no active geometry in the scene.



(a) Dithering and outline effects applied to cars (b) Associated ID visualisation

Figure 3.3: Stylised output and corresponding ID buffer representation

3.7 Bounding Box Prepasses

The bounding-box prepasses implement the core spatial localisation pipeline, organised into three sequential stages: generation, occlusion testing, and compaction. CPU and GPU variants are maintained as parallel execution paths and are never mixed within a single pipeline configuration; this separation is enforced by the renderer feature.

Generation

The generation stage constructs an initial set of screen-space bounding boxes for all stylised renderers visible within the camera frustum. At the start of each frame, the per-frame bounding-box state in `NprFrameData` is reset.
In the CPU path, each renderer's world-space bounds are projected into screen space using the near-plane clipping procedure described in subsubsection 2.5.1. Each valid box and its associated style mask are stored in a bounding-box data structure and appended to the CPU-side list in `NprFrameData`, an example is seen in Figure 3.4.
In the GPU path, the same information is uploaded to a compute shader which then executes two kernels. The first dispatches one thread per bounding box, each projecting the world-space input to a screen-space rectangle output. The second initialises the indirect draw argument buffer, which allows the GPU to dispatch a single instanced draw call covering all active regions without requiring the CPU to read back the region count. Each instance renders a screen-space quad of six vertices, with per-instance region data read from the rect and mask buffers by index.

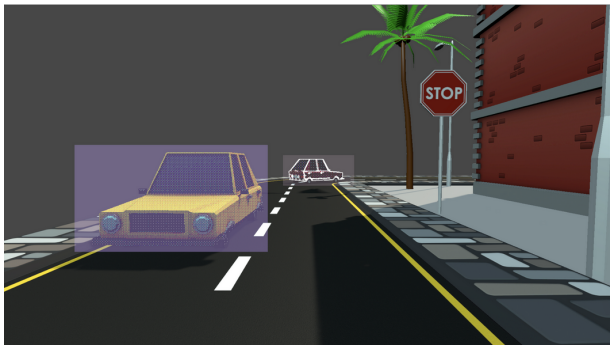


Figure 3.4: Visualisation of CPU/GPU bounding boxes

Occlusion Testing

The occlusion stage reads the ID texture and the region set from `NprFrameData` and writes a visibility buffer back into it as explained in subsubsection 2.5.1, which compaction and effect passes then consume. It refines the visibility buffer by scanning each bounding box region in the ID texture and marking as invisible any region containing no pixels matching the expected style mask as shown in Figure 3.5.

CPU path: Each bounding box is processed by an individual compute dispatch. Each dispatch scans the pixels within its assigned region and writes a binary result to the shared visibility buffer.

GPU path: All bounding boxes are processed within a single batched dispatch, with each thread group assigned to one bounding box. Threads within this group scan the region for the expected bitmask using a shared visibility flag. When a matching pixel is detected, the flag is set to one. The result is written to the visibility buffer once all threads in the group have completed.

A known limitation is that false positives occur when objects sharing a style mask overlap in screen space: a box may be incorrectly retained due to pixels from a different object carrying the same mask. This does not introduce visual artefacts but results in redundant computation in overlapping configurations.



Figure 3.5: Occluded Bounding Box Flagged

3.7.1 Compaction

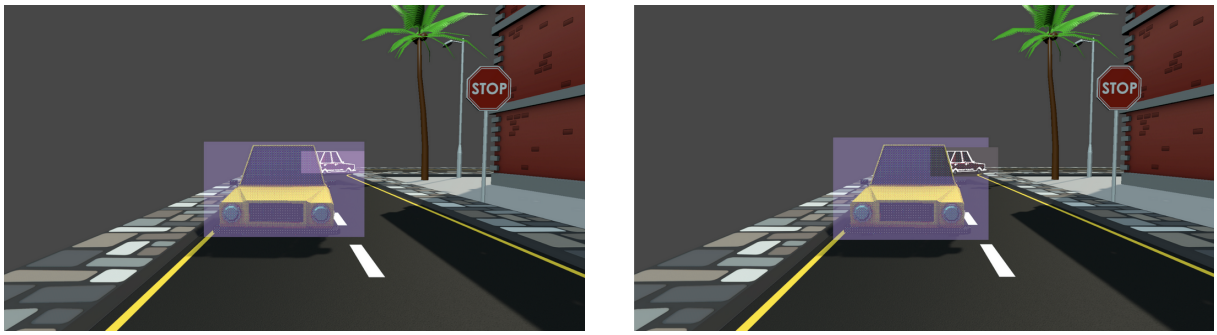
The compaction stage reads the region set and visibility buffer from `NprFrameData` and updates them with a reduced representation before the effect passes execute. This means effect passes always consume the same `NprFrameData` interface regardless of whether compaction has run.

CPU path: Compaction, as shown in Figure 3.6a, applies the pairwise merging algorithm described in subsubsection 2.5.1. Although no GPU work is performed, this stage is implemented as a `ScriptableRenderPass` so that it participates in the same scheduling, profiling, and toggling infrastructure as all other pipeline stages.

GPU path: The `GpuTiling` prepass executes three compute kernels: the first rasterises each visible bounding box into a tile mask buffer using an atomic OR operation; the second emits non-empty tiles into output rectangle, mask, and visibility buffers; the third rebuilds the indirect draw argument buffer with the tiles acting as the instances instead of the bounding boxes. The result of this is shown in Figure 3.6b.

3.8 Tiling Prepass

The tile generation prepass implements the standalone localisation strategy as described in subsection 2.5.2, deriving active regions directly from the ID texture. One thread is dispatched per tile and they accumulate the style masks at each contained pixel via bitwise OR. If the resulting mask is zero the tile contains no stylised pixels and is discarded; otherwise



(a) CPU compaction (b) GPU compaction

Figure 3.6: Comparison of compaction strategies with objects sharing the dithering effect but not the outline effect

the tile's screen-space rectangle and accumulated mask are written to output buffers via an atomic counter that assigns a unique index to each tile. A second kernel then writes the active tile count into the indirect draw argument buffer.

The tile size T is a configurable parameter set at pipeline initialisation. Using this, buffers are sized to the maximum possible tile count for the current screen resolution and tile size, ensuring no reallocation is required mid-frame. The visibility buffer is initialised to fully visible for all tiles, as any occluded objects cannot be found in the ID texture it operates on.



(a) Non-overlapping tiles (b) Overlapping tiles (c) Occluded tiles

Figure 3.7: Tile-based region generation under different spatial configurations ($T = 32$)

3.9 Source Prepass

A source prepass is enqueued once before each effect, capturing the current camera colour buffer into `NprFrameData` as the input texture for that effect's passes. This ensures each effect reads from the colour state produced by all preceding effects rather than the original scene colour, enabling correct sequential composition.

3.10 Effect Passes

Depending on the active rendering mode, the effect pass dispatches work as follows. In Fullscreen mode, a full-screen triangle is drawn and the fragment shader masks output using the ID texture. In CPU Localised mode, the GPU is instructed to restrict rasterisation to the bounding-box region before a full-screen triangle is drawn, so only fragments within that region are generated and reach the fragment shader. In GPU Localised mode, screen-space quads are drawn procedurally via an indirect draw call, with each instance reading its region rectangle

and style mask from the instance buffers by index. In both localised modes, occluded regions are handled in the vertex shader by collapsing their geometry to a degenerate point, producing no fragments and skipping rasterisation entirely. Tiling reuses the GPU path directly, treating tiles as fixed-size quad instances.

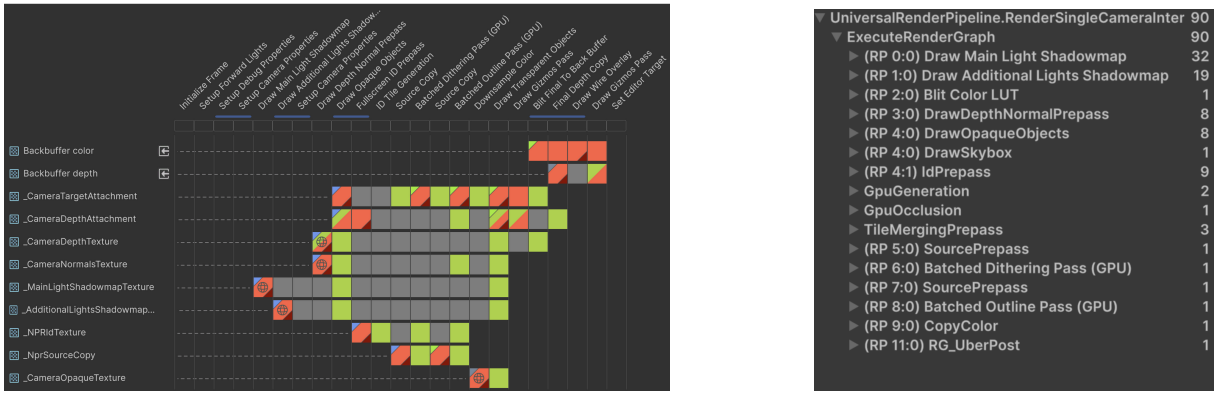
3.11 Validation

Validation was performed throughout development using six techniques. Each technique targets a different aspect of the system and together they provide coverage across all pipeline stages. Table 3.1 summarises which technique was applied to validate each system component.

Component	Validation Techniques Applied
Scriptable Renderer Feature	Frame Debugger, RenderGraph Viewer
Stylised Tag System	ID Visualisation
ID Prepass	ID Visualisation
Source Prepass	Frame Debugger
Bounding Box Generation	Region Overlay, Buffer Logging
Occlusion Testing	Occlusion Overlay, Buffer Logging
CPU Merging	Region Overlay, Buffer Logging
GPU Tile Compaction	Region Overlay, Buffer Logging
GPU Tile Generation	Region Overlay, Buffer Logging
Effect Passes	Output Validation

Table 3.1: Validation techniques applied to each pipeline component

Unity Debugging Tools: Unity's Frame Debugger, shown in Figure 3.8b provides the ability to step through the outputs of each render pass which was used to ensure they executed as expected [41]. The RenderGraph Viewer in Figure 3.8a, depicts the resources read from (green), written to (red), cleared (blue) or stored (brown) by each pass, used to validate that passes are accessing the resources they are expected to [43].



(a) RenderGraph viewer

(b) Frame Debugger view

Figure 3.8: Visualisation of the rendering pipeline using Unity debugging tools

ID Texture Visualisation: A debug mode was implemented in the ID prepass that hashes each unique style bitmask to a visually distinct colour. This gave the ability to validate that the correct stylisation bitmasks were assigned to each object.

Region Overlay: A debug overlay renders active regions directly to the screen, colour-coded by style mask. This was used to confirm screen-space projection accuracy, near-plane clipping behaviour, and viewport clamping for bounding-box generation, and correct screen-space coverage for tile generation.

Occlusion Overlay: A separate debug overlay colours each bounding box according to its computed visibility flag, distinguishing retained from discarded regions. This was used to validate occlusion testing under controlled configurations.

Buffer Logging: A dedicated debug class logs CPU-side list contents and dumps GPU structured buffer values to allow direct inspection of intermediate pipeline data. This was used to verify screen-space localisation structures and style masks against expected values after generation and/or compaction, as well as confirm visibility buffer correctness after occlusion testing.

Output Validation: To provide proof of consistency, a `Validator` component was implemented that compares the camera output of each rendering mode against the fullscreen baseline. Any pixel-level divergence above a threshold to account for renderer noise is flagged. This confirms that the rendering mode doesn't impact the visual result.

Chapter 4

Results, Evaluation and Discussion

4.1 Experimental Setup

All performance measurements were collected on the hardware described in Table C.1. Each run recorded 1000 frames following a 500-frame warm-up period, allowing shader compilation and resource allocation to stabilise. As the CPU and GPU execute concurrently, the mean, standard deviation, 95th percentile, and 99th percentile frame times were computed for both. Each test was carried out multiple times to reduce the influence of anomalous behaviour and the timings of each one averaged against each other. The most relevant timings are emphasised in this section, while the full CPU/GPU graphs are provided in Appendix D.

4.2 Results

The following tests evaluate each spatial localisation strategy against the fullscreen baseline by varying the independent variables and recording the metrics defined in section 2.7. The four base methods are compared across coverage, object count, style count, and overlap scenarios. Merging and occlusion are then evaluated as ablations on the bounding-box approaches, while tile size is evaluated as an ablation of the tiling method.

4.2.1 Number of Styles

In this test the impact of stacking N styles on a single object that covers 50% of the screen was evaluated. The results shown in Figure 4.1 support the hypothesis that spatial localisation methods positively impact performance in this scenario. Both the GPU and CPU bounding-box approaches perform comparably and outperform the fullscreen baseline across all values of N. This is evident from their consistently lower mean frame times, indicating a reduced baseline cost as well as improved scaling behaviour. In contrast, the tiling approach exhibits the highest baseline cost at low N, but demonstrates the most favourable scaling and stability characteristics. While it initially underperforms the other methods, it surpasses Fullscreen at $N \approx 4$ and GPU/CPU at approximately $N \approx 16$. Additionally, the relative tail metrics indicate that tiling maintains consistently lower frame-time variability compared to the other approaches, particularly at larger N, making it the most stable method under high style complexity.

4.2.2 Screen Coverage

The test investigates how the screen coverage of a single object with 32 styles applied impacts performance. The GPU results in Figure 4.2 indicate that both the tiling and fullscreen approaches scale linearly with a factor of $\sim$0.31, whilst bounding-boxed based approaches scale more drastically at $\sim$0.5. The positive impact of region-based stylisation reduces as the stylised region encompasses a greater proportion of the screen. Bounding Box approaches are outperformed by Tiling at approximately $\sim$40% coverage and then are effectively equal with

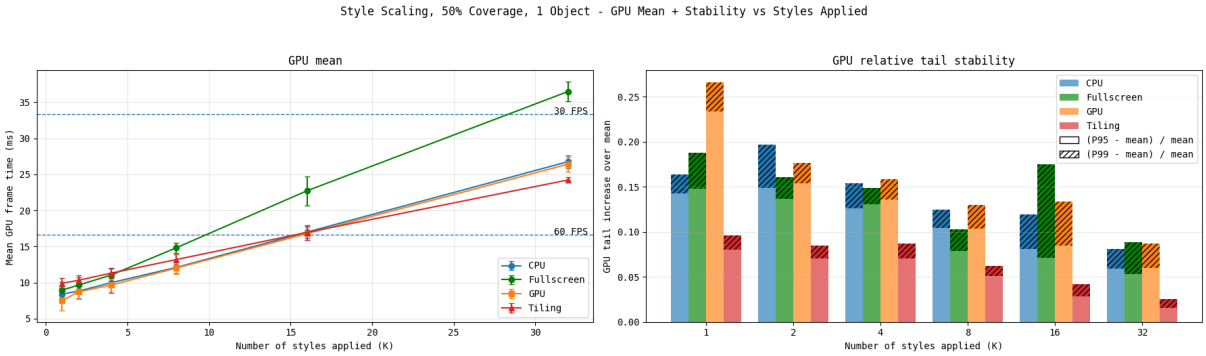


Figure 4.1: Impact of increasing number of styles on mean GPU frame time and stability

Fullscreen at 100% coverage. Although all methods process an equivalent number of pixels at full coverage, the tiling approach maintains lower cost and much more stable frame times, indicating that regular, fixed-size workloads utilise the GPU more efficiently than irregular bounding box dispatch. It is also worth noting that the high relative instability shown by the normalised tail graph for bounding-box approaches at 0% coverage is likely due to the mean time being so small that even small variations have a relatively large impact.

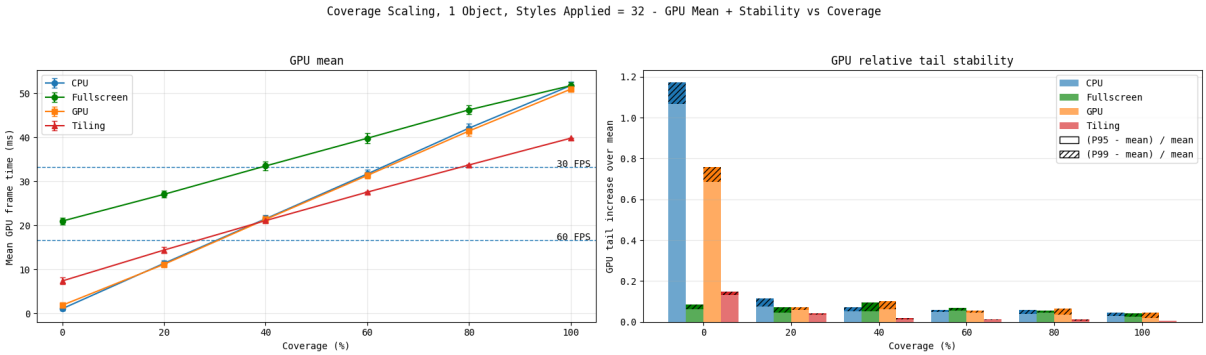
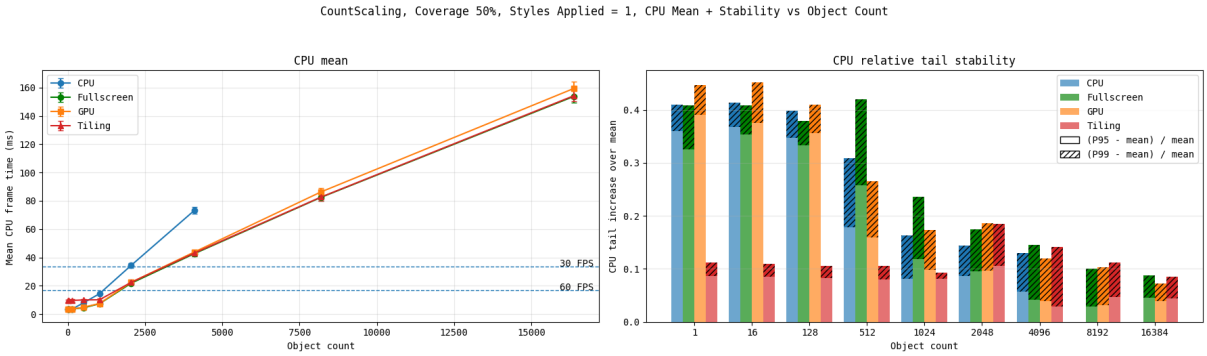


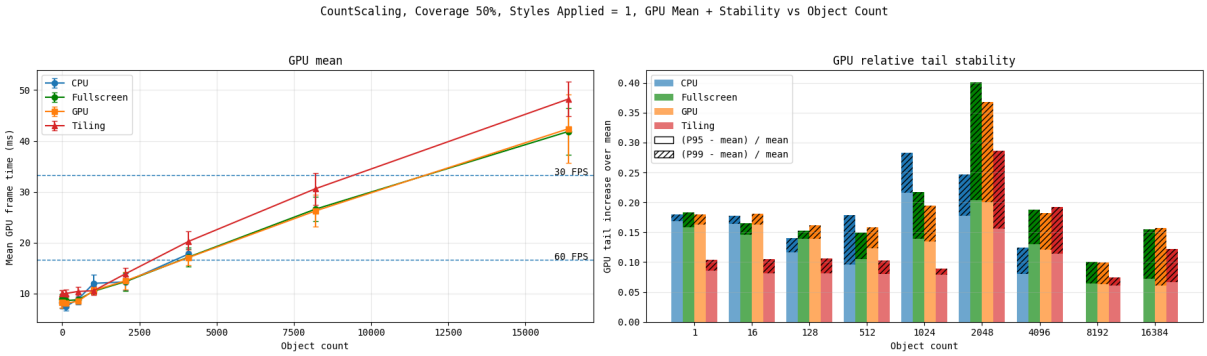
Figure 4.2: Effect of coverage scaling on mean GPU frame time and stability

4.2.3 Effect of increasing object count

This test evaluates the impact of applying 1 style to an increasing number of objects at a fixed 50% coverage. Figure 4.3a shows that Fullscreen, GPU, and Tiling scale near-identically and linearly with object count, suggesting that scene submission cost dominates over any per-method overhead at this scale. The CPU-based bounding-box method diverges sharply, as it computes and submits a separate render pass per bounding box, before crashing at beyond ∼4096 objects due to RenderGraph being unable to handle the excessive number of a render passes. Since pass count scales as objects × styles, this threshold falls proportionally as style count increases. Tiling exhibits a higher baseline cost but maintains significantly lower frame-time variance. As object count increases beyond ∼2048, its mean performance converges with Fullscreen while retaining superior stability. The GPU results in Figure 4.3b show similar scaling behaviour, though Tiling performs worst in terms of mean frame time. This is expected, as its workload is executed entirely on the GPU, however this cost is offset by improved stability, reinforcing that it produces a more consistent workload.



(a)



(b)

Figure 4.3: Effect of increasing object count on mean frame time and stability for (a) CPU and (b) GPU timings

4.2.4 Overlap Tests

This test starts with 100 objects with 32 styles applied that cover the full screen, they are shifted in by increasing amounts to introduce overlap. Figure 4.4 shows that Tiling and Fullscreen approaches scale much better as objects overlap each other. This is because as overlap increases the objects increasingly occupy the same regions of the screen and the total region to stylise decreases. Tiling and Fullscreen implicity exploit this as they both operate exclusively on the screen. In contrast, bounding-box approaches dispatch work per object. Since each bounding box processes its entire screen-space extent independently, regardless of whether other boxes cover the same pixels, overlapping objects lead to repeated shading of identical pixels and produces redundant work.

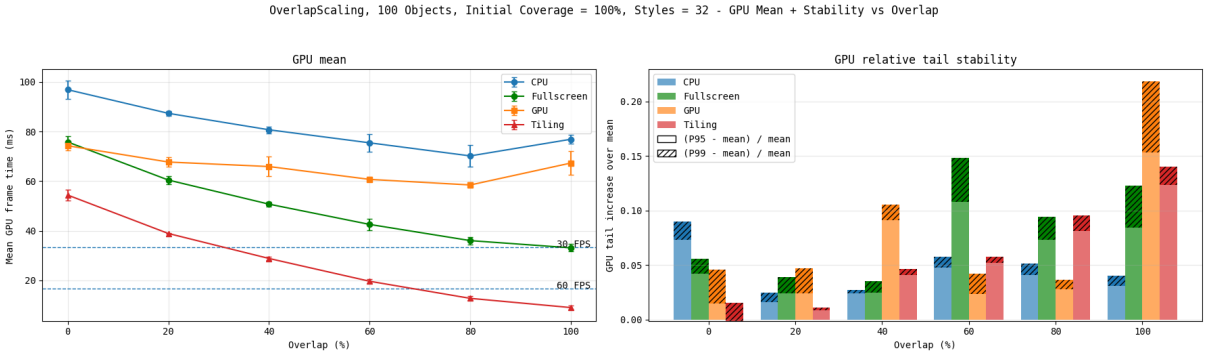
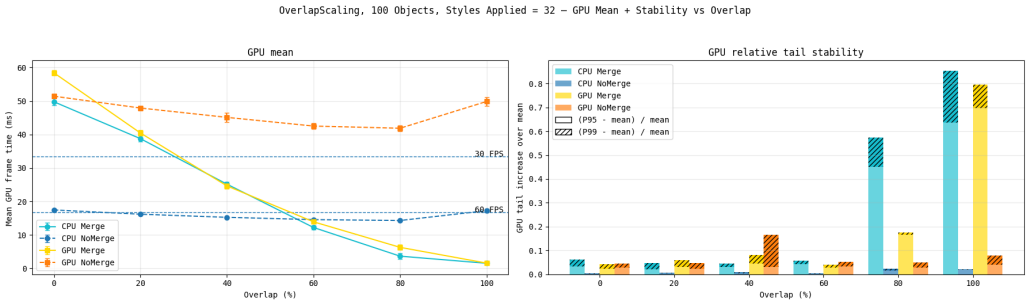


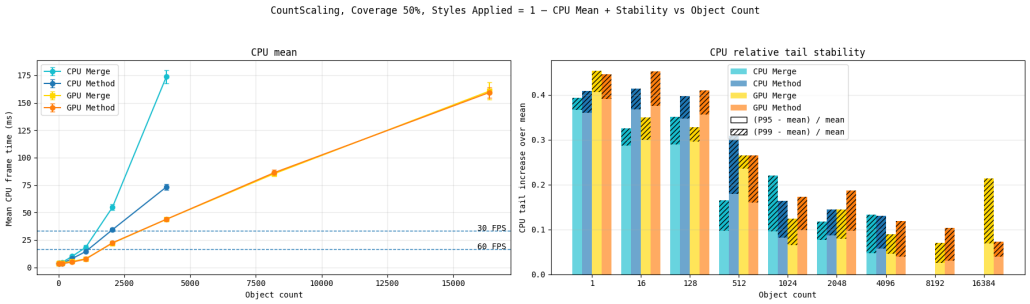
Figure 4.4: Effect of increasing overlap percentage on mean GPU frame time and stability

Merging Ablation

The overlap test was repeated for the bounding-box approaches with and without the compaction pass, as shown in Figure 4.5a. Compaction has a clear reduction in redundant shading when objects overlap with the GPU method benefiting from it at as little as ∼10% total overlap. Conversely, the CPU method only breaks even at roughly 55%, reflecting the higher cost of its CPU-side compaction step. This behaviour is consistent with the overlap model described in chapter 2, where the benefit of merging scales with the proportion of redundant screen-space coverage. However, merging is not unconditionally beneficial. To isolate the cost of merging itself, Figure 4.5b shows that under increasing object count with no overlap, GPU merging scales comparably to its non-merging counterpart, suggesting the compaction pass cost is roughly constant with respect to object count. The CPU merging implementation exhibits quadratic scaling as the CPU must perform pairwise overlap checks between all bounding boxes to identify merge candidates, making CPU merging unsuitable for scenes with large numbers of objects.



(a) Impact of merging on mean GPU frame time and stability (overlap test)



(b) Impact of merging on mean CPU frame time and stability (object count test)

Figure 4.5: Effect of merging under different scaling scenarios

4.2.5 Occlusion Test

When no occlusion culling is applied, bounding boxes occluded by foreground geometry are still dispatched and scanned for valid pixels, producing redundant work. Figure 4.6 shows how each base method responds to 100 objects covering 50% of the screen getting occluded by increasing amounts. Both CPU and GPU timings are shown as the methods are bound at different points: the CPU bounding-box method is CPU-bound due to per-box submission cost, whilst Fullscreen, GPU bounding-box, and Tiling are GPU-bound. Tiling shows the greatest improvement with occlusion, falling from ∼25 ms to ∼12 ms on the GPU, because it operates

on the ID buffer produced from the camera view, occluded objects are implicitly ignored. The GPU bounding-box method improves moderately but cannot fully exploit occlusion as its passes are dispatched from world-space bounds, with no mechanism to detect that a region contains no visible stylised pixels. The CPU method remains flat on the CPU side at ∼20 ms for the same reason. These results highlight that, without explicit culling, bounding-box approaches are unable to exploit occlusion effectively and thus motivates the occlusion techniques evaluated in the following section.

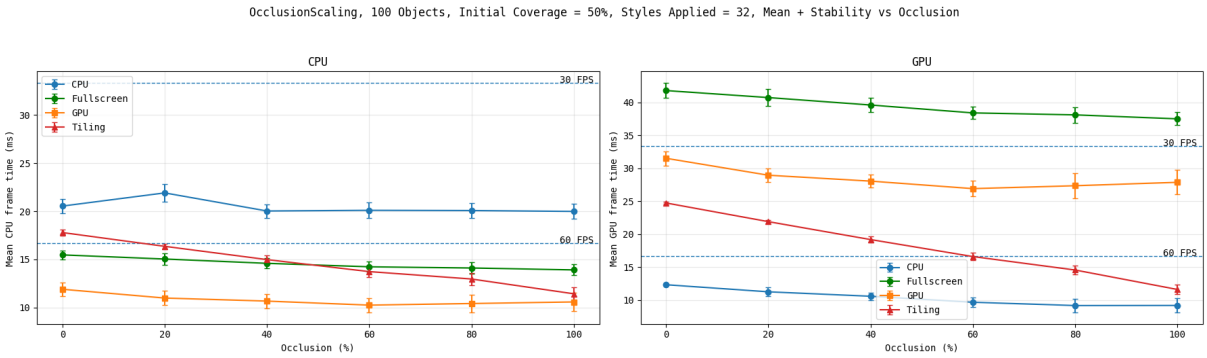


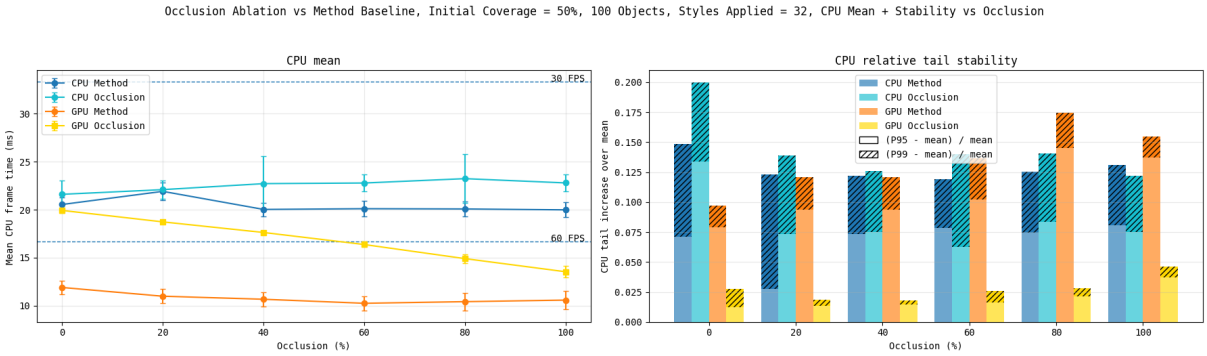
Figure 4.6: The impact of increasing occlusion percentage on mean CPU and GPU frame times

Occlusion Ablation

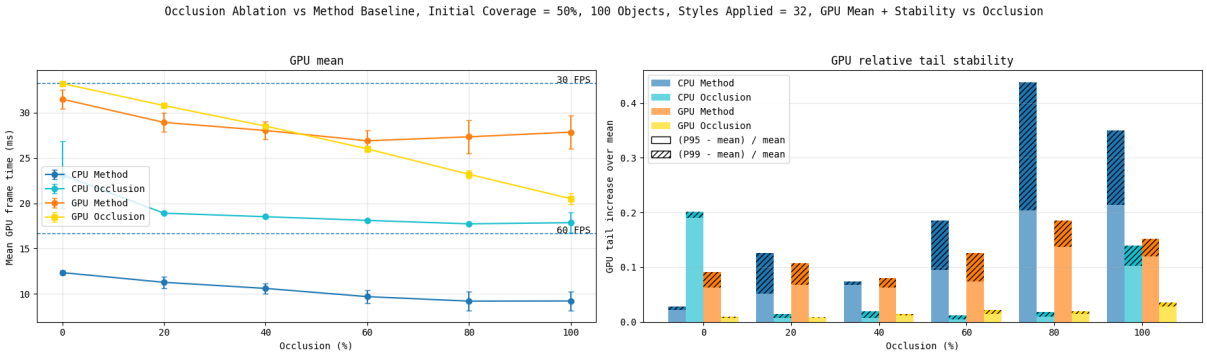
Figure 4.7a compares the bounding-box approaches with and without occlusion culling. The CPU-based implementation exceeds the cost of the redundant fragment work it eliminates and therefore fails to outperform the base CPU method. When comparing GPU occlusion against the base method in Figure 4.7b, the occlusion implementation exhibits performance increases with occlusion percentage, albeit with a higher baseline frame time. At ∼50% the benefit of occlusion outweighs the overhead and it begins outperforming the base method. Notably, both occlusion methods exhibit very stable timings on the GPU, suggesting that occlusion culling produces a more predictable execution pattern. The GPU occlusion method also exhibits improved CPU-side stability, reflecting the reduced number of region submissions.

4.2.6 Effect of Tile Size

The base tiling method uses 32×32 pixel tiles. Tile size introduces a trade-off between spatial precision and overhead where smaller tiles more accurately localise work but increase the number of regions that must be processed. Figure 4.8a shows that smaller tile sizes incur a significant GPU cost penalty as object count increases with 32 styles applied over 50% screen coverage. The 8×8 tiles begin at ∼38ms versus ∼26 ms for the larger sizes, and diverge further to ∼67 ms at 16264 objects. This is consistent with the up to 16× increase in tile count. GPU timings are shown as render command submission cost dominates the CPU graph, compressing differences between tile sizes. However, smaller tiles become advantageous when objects are small relative to the tile footprint; as shown in Figure 4.8b. At the smallest object scales, 64×64 tiles plateau at ∼20 ms whilst 8×8 and 16×16 tiles fall to ∼8 ms, as larger tiles waste significant work on empty regions when objects occupy only a fraction of their footprint. Smaller tiles show greater variance at the smallest object scales which is indicative of reduced



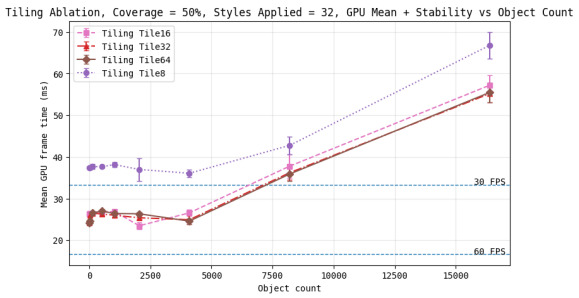
(a) CPU timings



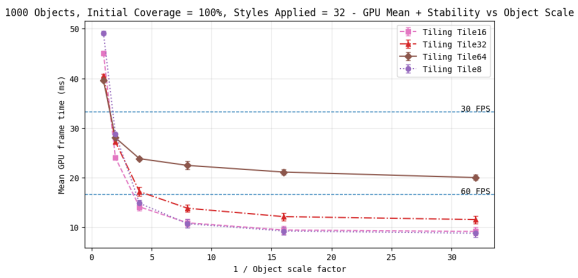
(b) GPU timings

Figure 4.7: Effect of occlusion culling on mean frame time and stability for (a) CPU and (b) GPU

spatial coherence, the degree to which neighbouring tiles exhibit similar behaviour, as the small tiles will alternate between active and inactive across the screen.



(a) Effect on mean GPU frame time of increasing object count with different tile sizes



(b) Effect on mean GPU frame time of decreasing object scale with different tile sizes

Figure 4.8: Impact of tile size under different scene configurations

4.3 Conclusions

The objectives set out in subsection 1.1.1 have been addressed through the implementation and evaluation of the multi-style NPR pipeline. It supports multiple concurrent effects, localises work using 3 distinct methods and evaluates the cost associated over the variables defined in section 2.7. As identified in chapter 1, applying multiple NPR effects as fullscreen passes leads to inefficient computation when stylisation is only required for part of the screen. In such

cases, rendering cost scales with screen resolution and increases with each additional effect, which is not well suited to real-time systems. The results support that spatial localisation addresses this issue, by restricting computation to regions containing stylised pixels both redundant fragment processing and the rate at which cost grows with additional styles are reduced. Screen coverage is the dominant factor governing this benefit, with substantial gains at low coverage that diminish as stylised regions approach fullscreen. Different localisation strategies exhibit distinct trade-offs. Bounding box approaches provide strong performance at low coverage and object counts but degrade as overlap increases due to redundant region processing and scale worse with scene complexity due to the reliance on the CPU to obtain the bounds for each object. In contrast, tiling has a higher baseline cost but offers better scaling and more stable frame times. This suggests that regular, spatially coherent workloads lead to more efficient and stable execution on the GPU. In conclusion, the results indicate that spatial localisation is most scalable when rendering work is structured to exploit GPU parallelism, and that the choice of strategy should be driven by scene characteristics rather than a single universal approach.

4.4 Future Work

As the tests were all carried out on single system they lack generality and would benefit from repeated runs on different machines. Unity also does not expose per-pass GPU timing or Video Random Access Memory (VRAM) on the system. so evaluation relies on CPU and GPU frame times rather than per-pass timings which would provide greater insight. As the implementation makes significant reuse of render targets the memory statistics are not expected to be vital to the evaluation but would still be worth investigating for a more comprehensive result. Additionally, the test scenes and uniform test effects, whilst enabling controlled comparison, may not reflect realistic real-time rendering scenarios found in dynamic video games. A controlled scene flythrough would be a valuable test scenario to extend the evaluation.

4.4.1 Effects Extending Beyond the ID Buffer

The current implementation restricts stylisation to regions defined by the ID texture, meaning effects that require sampling outside object boundaries cannot be supported directly. Watercolour bleed, where pigment diffuses beyond object edges onto surrounding regions [7], is a prominent example. Local segmentation strategies, as discussed in subsection 1.2.3, offer a route to supporting such effects within a localised pipeline, though the additional per-pixel overhead would need to be evaluated. An alternative approach would be inspired by the clustered shading approach mentioned in subsection 1.2.5, which partitioned the view frustum into a three-dimensional grid. This may be applicable with the ID pass done by this system, where each cluster holds the stylisation masks contained at a given depth. This would allow effects to distinguish between foreground and background surfaces when sampling beyond object boundaries and avoiding artefacts where they bleed onto objects that should occlude them. Storing and processing multi-layer data per region would introduce both increased memory overhead to hold the representation and extra work looking them up in the stylisation passes, so these performance costs would again need to be investigated thoroughly.

4.4.2 Adaptive Tiling

The fixed tile size used in the tiling method is a tradeoff where small tiles waste work in dense scenes whilst large tiles waste work on small objects, and the baseline cost is fixed regardless of scene content. A data structure, such as a quadtree, built over the ID texture each frame could address this by adaptively subdividing screen space, discarding tiles containing no tagged pixels and dispatching work only where stylised objects are present. As the quadtree adapts its granularity based on scene content, it can accurately represent both large densely stylised regions and small sparse objects within the same frame, without the fixed tile size tradeoff that requires manual tuning for different scene types. The biggest drawback however is that the adaptive structure of quadtrees is not suited towards GPU execution. This could be overcome by approximating it with precomputed region sizes to find a middle ground between adaptibility and GPU efficiency. This adaptive structure could also be applied to the clustered partitioning mentioned earlier.

References

[1] T. Akenine-Möller, E. Haines, and N. Hoffman. *Real-Time Rendering*. A K Peters/CRC Press, 4 edition, 2018.

[2] D. M. Ansari, D. Kurchaniya, and M. Dixit. A comprehensive analysis of image edge detection techniques. *International Journal of Multimedia and Ubiquitous Engineering*, 12:1–12, 11 2017.

[3] Autodesk, Inc. Autodesk maya, 2024. 3D computer graphics software. Accessed: 2026-04-13.

[4] P. Barla, J. Thollot, and L. Markosian. X-toon: an extended toon shader. In *Proceedings of the 4th International Symposium on Non-Photorealistic Animation and Rendering*, NPAR '06, page 127–132, New York, NY, USA, 2006. Association for Computing Machinery.

[5] Blender Foundation. Compositing - blender manual, 2024. Accessed: 2026-04-13.

[6] A. Bleron. *Real-time stylized rendering of 3D animated scenes*. Theses, Université Grenoble Alpes, Nov. 2018.

[7] C. J. Curtis, S. E. Anderson, J. E. Seims, K. W. Fleischer, and D. H. Salesin. Computer-generated watercolor. In *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '97, page 421–430, USA, 1997. ACM Press/Addison-Wesley Publishing Co.

[8] M.-A. Davignon and B. St. Clair. Non-photorealistic compositing of the spider-verse: Showing the hand of the artist without painting every pixel. In *ACM SIGGRAPH 2023 Talks*, SIGGRAPH '23, New York, NY, USA, 2023. Association for Computing Machinery.

[9] J. de Vries. Deferred shading, n.d. Accessed: 2026-05-03.

[10] G. Denes, A. Jindal, A. Mikhailiuk, and R. K. Mantiuk. A perceptual model of motion quality for rendering with adaptive refresh-rate and resolution. *ACM Trans. Graph.*, 39(4), Aug. 2020.

[11] J. Dos Santos, J. K. Thompson, and K. Powers. Spider-man: Across the spider-verse. Film, 2023.

[12] W. et al. Non-photorealistic rendering of 3d point clouds using segment-specific image-space effects. Technical report, Hasso-Plattner-Institute, University of Potsdam, Germany, 2024.

[13] J. Friedman and A. C. Jones. Fully automatic id mattes with support for motion blur and transparency. In *ACM SIGGRAPH 2015 Posters*, SIGGRAPH '15, New York, NY, USA, 2015. Association for Computing Machinery.

[14] A. Gooch, B. Gooch, P. Shirley, and E. Cohen. A non-photorealistic lighting model for automatic technical illustration. *Proceedings of the ACM SIGGRAPH Conference on Computer Graphics*, 09 1999.

[15] B. Gooch and A. Gooch. *Non-Photorealistic Rendering*. A K Peters/CRC Press, 07 2001.

[16] M. Harris and S. Hargreaves. Deferred shading. NVIDIA Developer Presentation, 2004.

[17] S. Hegde, C. Gatzidis, and F. Tian. Painterly rendering techniques: a state-of-the-art review of current approaches. *Computer Animation and Virtual Worlds*, 24(1):43–64, 2013.

[18] A. Hertzmann. Tutorial: A survey of stroke-based rendering. *IEEE Comput. Graph. Appl.*, 23(4):70–81, July 2003.

[19] IGN. The legend of zelda: Breath of the wild review, 2017. Accessed: 2026-05-05.

[20] L. Istead, J. Istead, A. Pocol, and C. S. Kaplan. A simple, stroke-based method for gesture drawing. *Virtual Reality and Intelligent Hardware*, 4(5):381–392, 2022. Computer graphics for metaverse.

[21] Khronos Group. glscissor - opengl 4 reference pages, 2023. Accessed: 2026-02-24.

[22] Khronos Group. Opengl, 2023. Accessed: 2026-02-24.

[23] A. Kolliopoulos, J. M. Wang, and A. Hertzmann. Segmentation-based 3d artistic rendering. In *Proceedings of the 17th Eurographics Conference on Rendering Techniques*, EGSR '06, page 361–370, Goslar, DEU, 2006. Eurographics Association.

[24] J. E. Kyprianidis, J. Collomosse, T. Wang, and T. Isenberg. State of the "art": A taxonomy of artistic stylization techniques for images and video. *IEEE Transactions on Visualization and Computer Graphics*, 19(5):866–885, 2013.

[25] J. Lext, U. Assarsson, and T. Moller. A benchmark for animated ray tracing. *Computer Graphics and Applications, IEEE*, 21:22 – 31, 04 2001.

[26] P. Litwinowicz. Processing images and video for an impressionist effect. In *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '97, page 407–414, USA, 1997. ACM Press/Addison-Wesley Publishing Co.

[27] S. Liu, A. Kuwahara, J. J. Scovell, and M. Claypool. The effects of frame rate variation on game player quality of experience. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, New York, NY, USA, 2023. Association for Computing Machinery.

[28] Lucas Pope. Return of the Obra Dinn, 2018.

[29] S. Montesdeoca, H. S. Seah, D. Benvenuti, P. Bénard, H.-M. Rall, J. Thollot, and R. Vergne. Direct 3d stylization pipelines. In *ACM SIGGRAPH 2017 Real Time Live!*, SIGGRAPH '17, page 18, New York, NY, USA, 2017. Association for Computing Machinery.

[30] Nintendo. The legend of zelda: Breath of the wild, 2017. Video game, Nintendo Switch and Wii U.

[31] O. Olsson and U. Assarsson. Tiled shading. *Journal of Graphics*, GPU:235–251, 11 2011.

[32] O. Olsson, M. Billeter, and U. Assarsson. Clustered deferred and forward shading. *High-Performance Graphics 2012, HPG 2012 - ACM SIGGRAPH / Eurographics Symposium Proceedings*, pages 87–96, 01 2012.

[33] B. Persichetti, P. Ramsey, and R. Rothman. Spider-man: Into the spider-verse, 2018.

[34] N. Redmond and J. Dingliana. Investigating the effect of real-time stylisation techniques on user task performance. In *Proceedings of the 6th Symposium on Applied Perception in Graphics and Visualization*, APGV '09, page 121–124, New York, NY, USA, 2009. Association for Computing Machinery.

[35] Sony Pictures Animation, Inc. Generating a look of characters using visible brush strokes, Dec. 2021. US Patent.

[36] M. Spindler, N. Röber, R. Döhring, and M. Masuch. Enhanced Cartoon and Comic Rendering . In D. Fellner and C. Hansen, editors, *EG Short Papers*. The Eurographics Association, 2006.

[37] A. Treviño and A. Maamar. Blender 2.81 splash screen (cryptomatte view), 2019. Rendered 2020. Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Accessed: 2026-04-24.

[38] R. Ulichney. A review of halftoning techniques. *Proceedings of SPIE - The International Society for Optical Engineering*, 3963, 02 2000.

[39] Unity Technologies. Unity Asset Store Terms of Service and EULA, 2024. Last updated: 4 December 2024. Accessed: 2026-05-04.

[40] Unity Technologies. Unity terms of service, 2025. Accessed: 2026-05-01.

[41] Unity Technologies. Introduction to the frame debugger, 2026.

[42] Unity Technologies. Unity, 2026. Game development platform.

[43] Unity Technologies. Use the render graph viewer. `https://docs.unity3d.com/6000.0/Documentation/Manual/urp/render-graph-view.html#use-the-render-graph-viewer`, 2026. Accessed: 12-04-2026.

[44] S. Verlander, E. Offertaler, A. Lee, B. Van Opstal, J. Mahovsky, and J. Budsberg. Creating a 2d data format capable of handling complex transparencies. In *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks*, SIGGRAPH Talks '25, New York, NY, USA, 2025. Association for Computing Machinery.

[45] H. Winnemöller. Xdog: advanced image stylization with extended difference-of-gaussians. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Non-Photorealistic Animation and Rendering*, NPAR '11, page 147–156, New York, NY, USA, 2011. Association for Computing Machinery.

Appendix A

Self-appraisal

A.1 Critical self-evaluation

Overall, I am proud of the result of my efforts over the course of this project. The choice of an agile project management style was definitely well suited to this project as it enabled me to adapt to the experimental nature of the project. By approaching spatial optimisations incrementally with a set of validation tools, I reduced the risk associated with this experimental problem. The agile approach also allowed me to continue building upon my knowledge of the problem area as I developed the solution, which is what led to the implementation of the Tiling method.

In hindsight, this may have come about sooner if more time had been put into the background research before implementation. My initial exploration of the problem space was not sufficiently broad, and focused more on how to implement individual NPR effects rather than evaluating different approaches to structuring and optimising their execution. This meant that a significant portion of early development time was spent focusing on what I was building rather than why I was building it. A clear problem statement only emerged after multiple image-space effects had been implemented that, while useful for understanding shader development, were not directly aligned with the eventual focus of the project on performance and spatial localisation. The incremental development approach did help mitigate the impact of this early misdirection as it allowed the project to be quickly refocused once the problem had been more clearly established.

The benchmarking aspect of this project is an area that should have been approached a lot more rigorously from the beginning. The initial benchmarks that were created were not scientifically rigorous and had to be drastically reworked which resulted in benchmarking not being completed until significantly later than would've been ideal. The use of a more structured benchmarking methodology allowed for a more systematic assessment of the different aspects of the solution. However, the evaluation remains limited by its reliance on static test scenes with aggregated frame times over dynamic scenes with frame-by-frame analysis. While these allow for controlled comparison between methods, they do not fully capture the performance in a realistic solution. Another limitation that, in hindsight, could likely have been identified earlier through more extensive background research was the restricted support for low-level GPU benchmarking and profiling tools on Apple devices within Unity. This constrained the level of performance analysis that could be carried out, particularly with respect to detailed GPU timing information and memory usage.

Despite these limitations, I think that the project demonstrates a well-structured exploration of spatial localisation techniques. The iterative development process, combined with targeted validation and eventual rigorous benchmarking, ensured that the final system was both functionally correct and meaningfully evaluated.

A.2 Personal reflection and lessons learned

This project has taught me a lot in terms of how I approach large-scale research problems. The jump from the clear, well-defined coursework projects I was comfortable with to the open-ended nature of this project was very intimidating at the start. I experienced a degree of impostor syndrome when struggling to understand research papers, which combined with a reluctance to seek help out of fear of appearing incompetent, resulted in my research stalling early on. The most important lesson I learned from this is that research is inherently an iterative process, and that being comfortable with uncertainty, failure, and spending time exploring possible solutions is valuable. This is a very different mindset from the way I approached coursework, where problems typically have clear solutions and objectives. Discussions about the project became significantly more valuable once I began to view them as opportunities to refine ideas rather than tests of competence.

The larger time scale also has made me reevaluate my usual method of short periods of intense effort close to deadlines, as it did not translate effectively to this continuously evolving project. I found that I struggled to focus on researching early on since the deadline was much further away than anything I had worked on before. This lack of focus was especially evident in the writing process. In hindsight, I should have placed much greater emphasis on structuring the report and producing draft sections earlier in the project. Much of my understanding of the system only became fully clear once I attempted to formally explain and justify it in writing. Earlier drafting would therefore not only have improved time management, but also helped refine the scope and direction of the project sooner.

Over the course of this project, my mindset shifted significantly. At the beginning, I had a lot of doubts about both my own contributions and my ability to work within the scope of research. Now, I am more confident in how I will approach similar projects in the future and I have truly been enjoying the exploratory aspect of graphics programming. I look forward to attempting to implement the future work items (after a little bit of a break!). I only regret that I didn't reach this mindset earlier in the course of this project as it has allowed me to engage much more deeply with the work I have done.

In conclusion, this dissertation has significantly improved both my technical confidence and ability to approach open-ended research problems. It has reinforced the importance of combining structured planning and communication with iterative experimentation, and has made me much more comfortable with trying and failing in order to learn.

A.3 Legal, social, ethical and professional issues

A.3.1 Legal issues

As this project does not process or store personal data the GDPR/data-protection concerns are minimal. It does however use external tools and assets such as *Unity* and *Free Cartoon World* which requires it to comply with their associated software licenses. This is particularly important if the software is redistributed or released publicly. Proper attribution and citation of all external material was also an important consideration.

A.3.2 Social issues

The optimisations provided by this project are partially motivated by reducing the hardware requirements associated with computationally expensive rendering techniques. By improving rendering efficiency, stylised rendering may become available to a wider array of devices and as such a greater range of users. This could aid in reducing technical barriers between artistic direction and real-time implementation in media like video games.

A.3.3 Ethical issues

The ethical risks of this project are limited as it does not involve, surveillance, AI, sensitive data or autonomous systems. The rendering system itself is designed to be generalised and not linked to specific NPR effects or applications. As such, concerns may arise from how a final application were to use this architecture rather than the solution itself.

A.3.4 Professional issues

The professional issues raised in this project mainly centre on aligning with the principles from the British Computer Society (BCS) Code of Conduct regarding professional integrity in software development. A transparent benchmarking methodology and explicit acknowledgement of limitations in the evaluation were used to avoid misleading conclusions or overstating the performance benefits of the proposed techniques. During development a maintainable, modular design was followed as in a professional setting the structuring of complex systems in a clear, extensible manner is very important.

Appendix B

External Material

Material	**Use in project**
Unity Engine and Universal Render Pipeline (URP)	Used as the underlying rendering engine and framework for implementation of the NPR pipeline. Usage was subject to the Unity Editor Software Terms and Unity Companion License [40].
Free Cartoon World from the Unity Asset Store	Used as pre-existing 3D scene content for visual testing and screenshots. The asset was not modified beyond placement/configuration in Unity scenes. It was licensed under the Unity Asset Store Terms of Service and EULA [39].

Table B.1: External material used in the project

Appendix C

Specs

Component	Specification
CPU	Apple M1
L1 Data Cache (per core)	128 KB
L2 Cache (per core)	12 MB
System RAM	8 GB LPDDR4X @ 4266 MT/s
Operating System	macOS 15.02
Unity Version	6000.3.2f1
URP Version	17.2.0
Display	1440 x 900

Table C.1: Hardware and software configuration used for all performance evaluations

Appendix D

Full Graphs

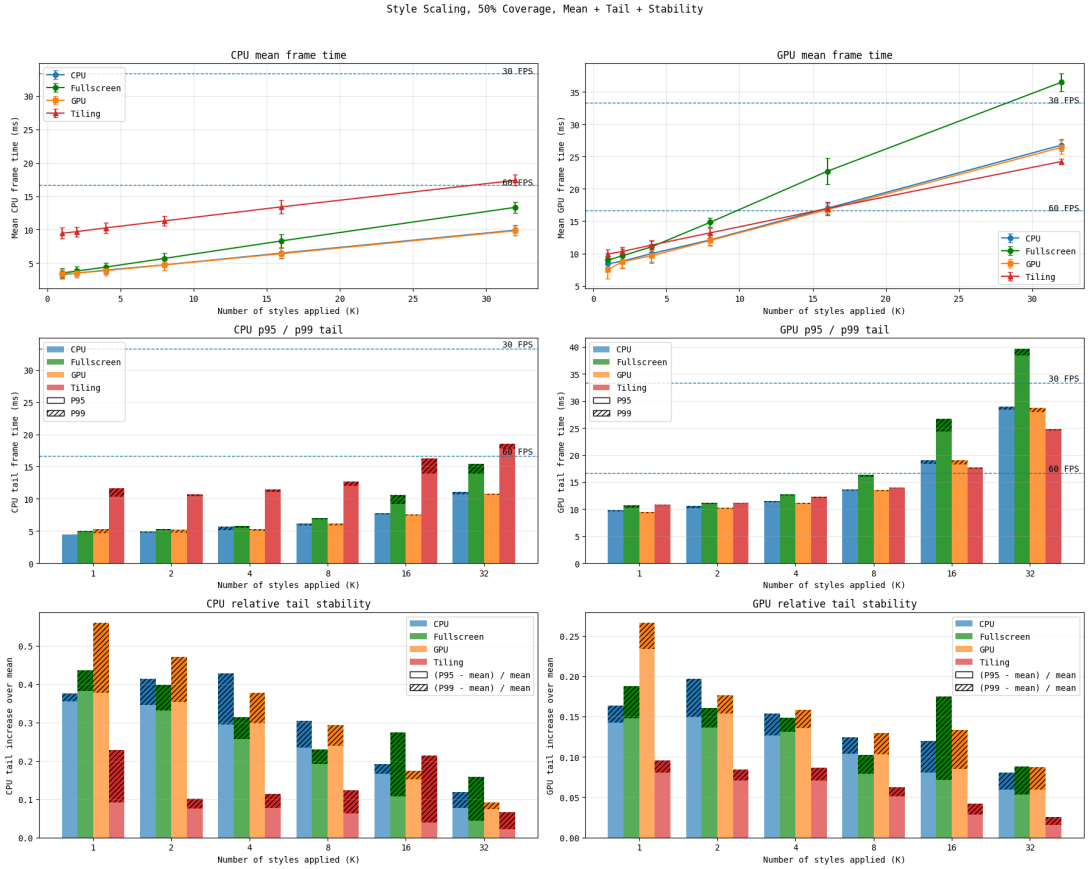


Figure D.1: Mean Frame Time, Tail and Stability Graphs for the Style Scaling Test

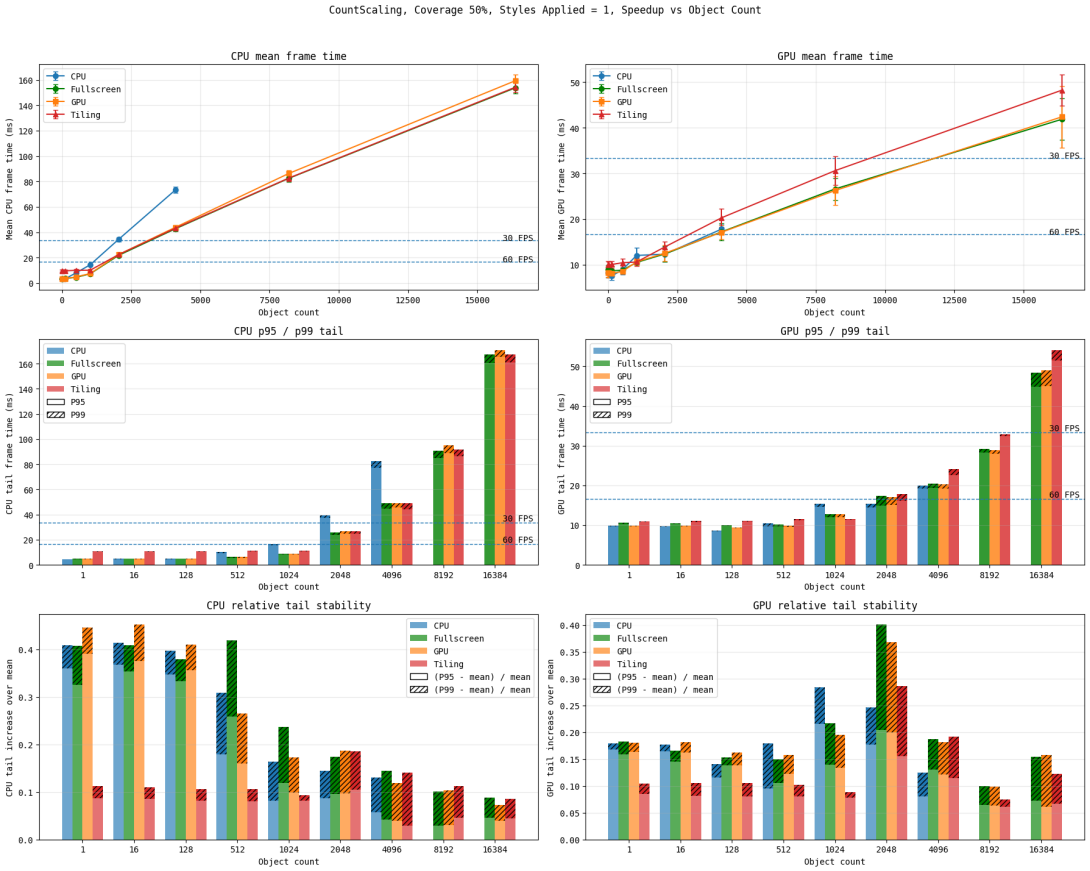


Figure D.2: Mean Frame Time, Tail and Stability Graphs for the Object Count Scaling Test

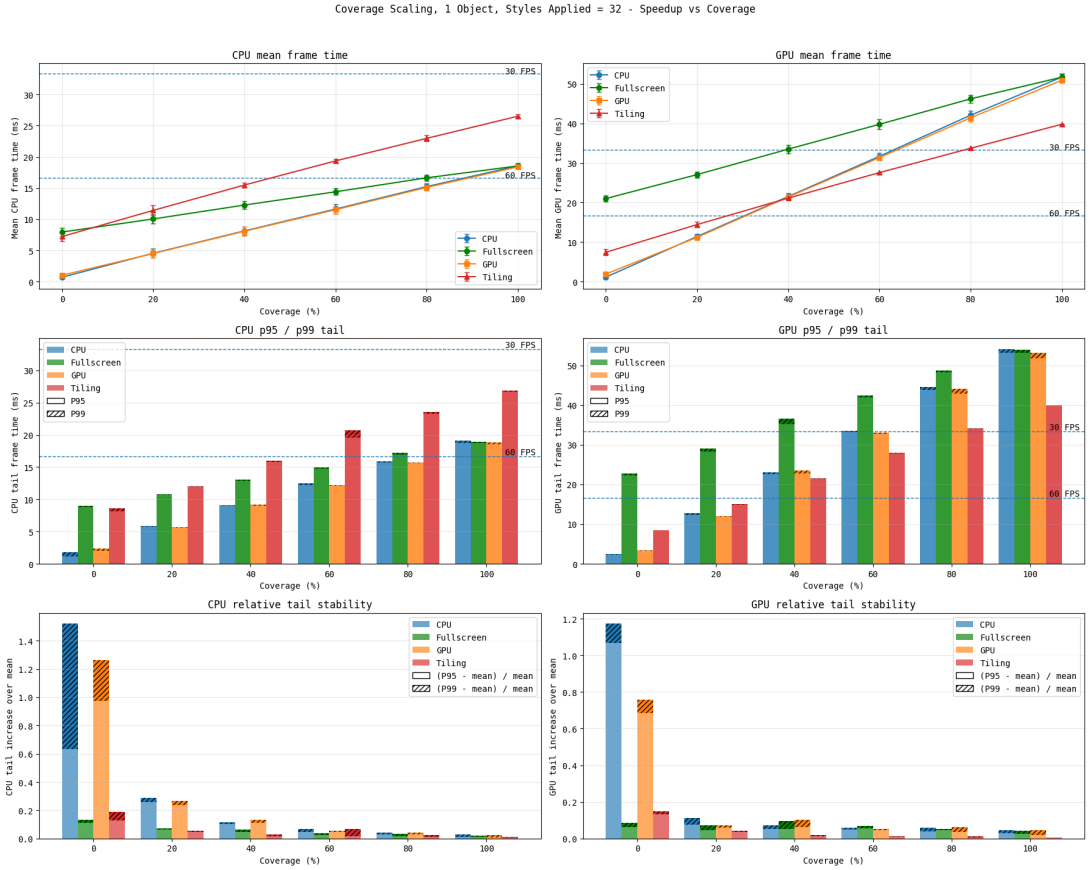


Figure D.3: Mean Frame Time, Tail and Stability Graphs for the Coverage Scaling Test

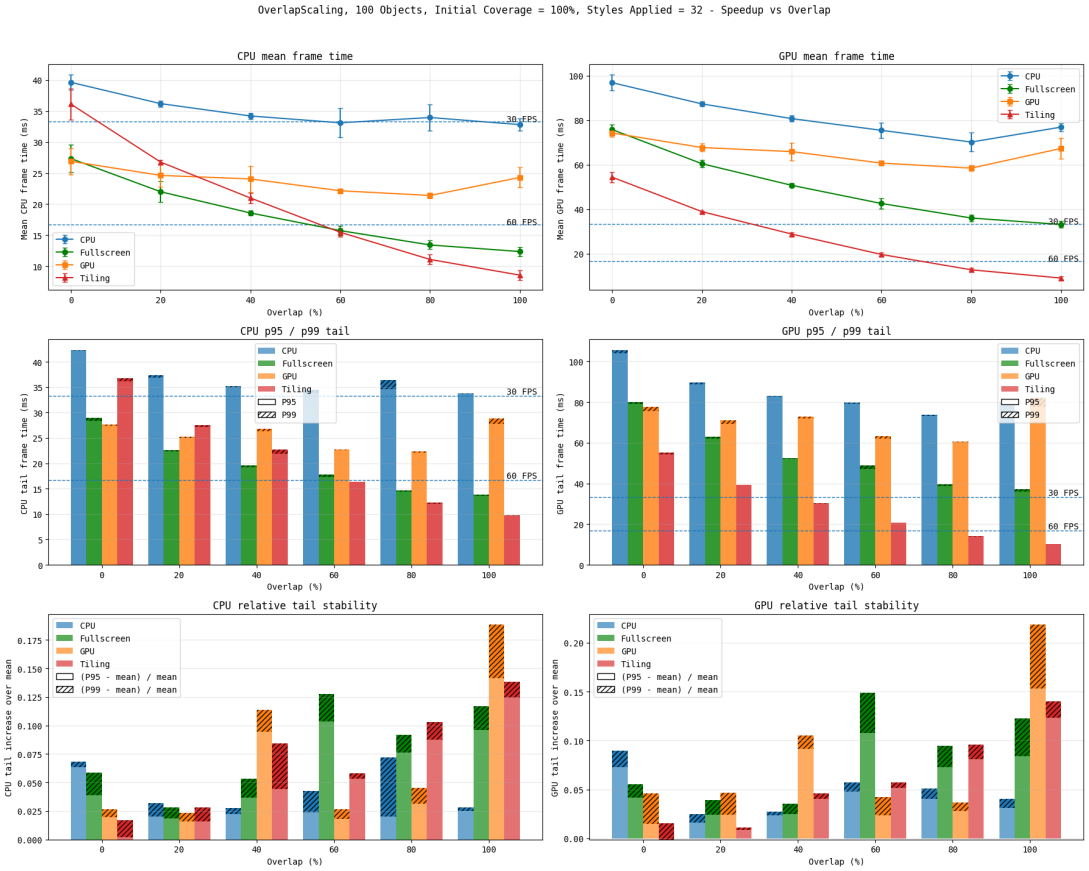


Figure D.4: Mean Frame Time, Tail and Stability Graphs for the Overlap Scaling Test

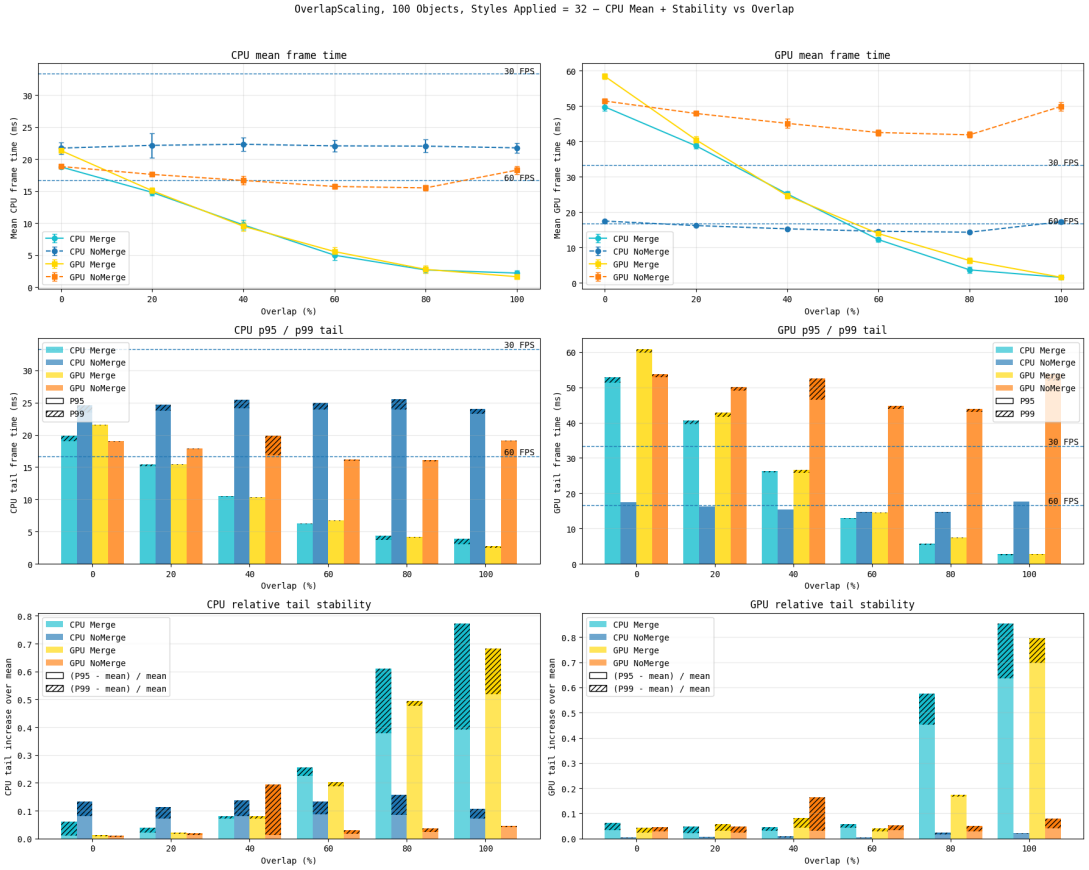


Figure D.5: Mean Frame Time, Tail and Stability Graphs for the Overlap Scaling Merge Ablation

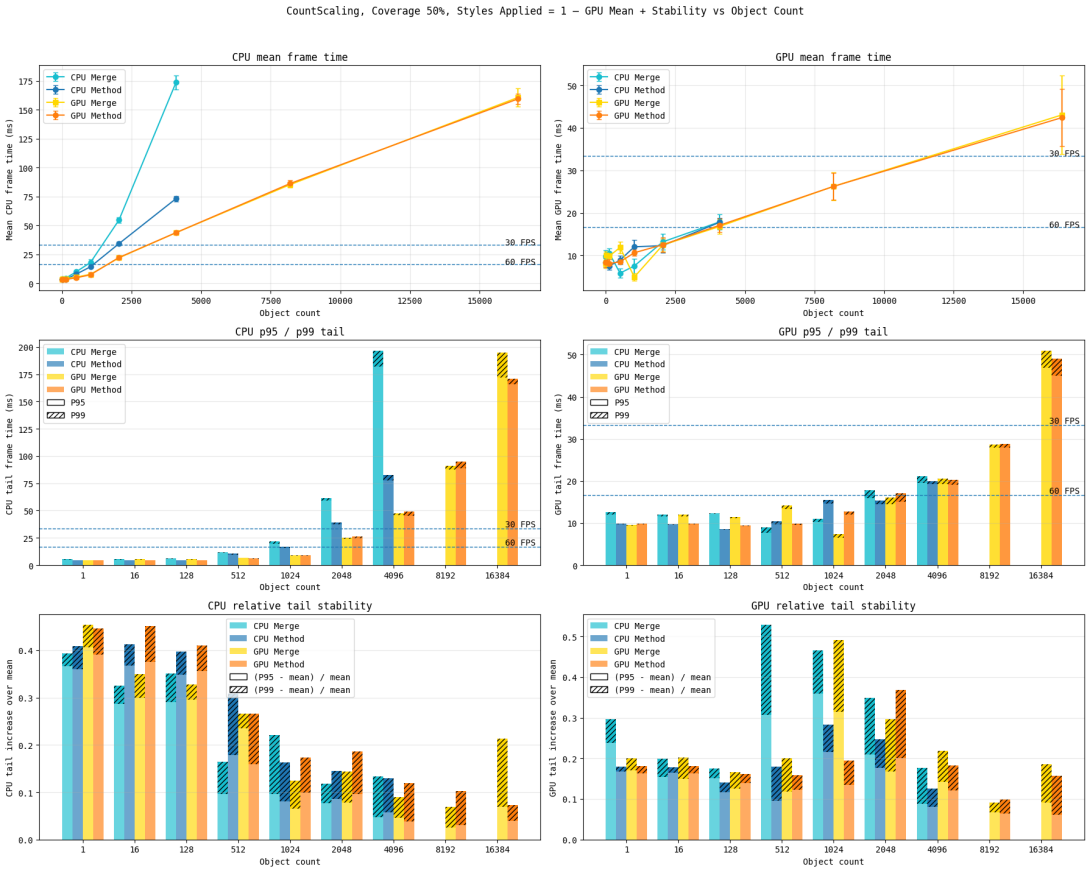


Figure D.6: Mean Frame Time, Tail and Stability Graphs for the Count Scaling Merge Ablation

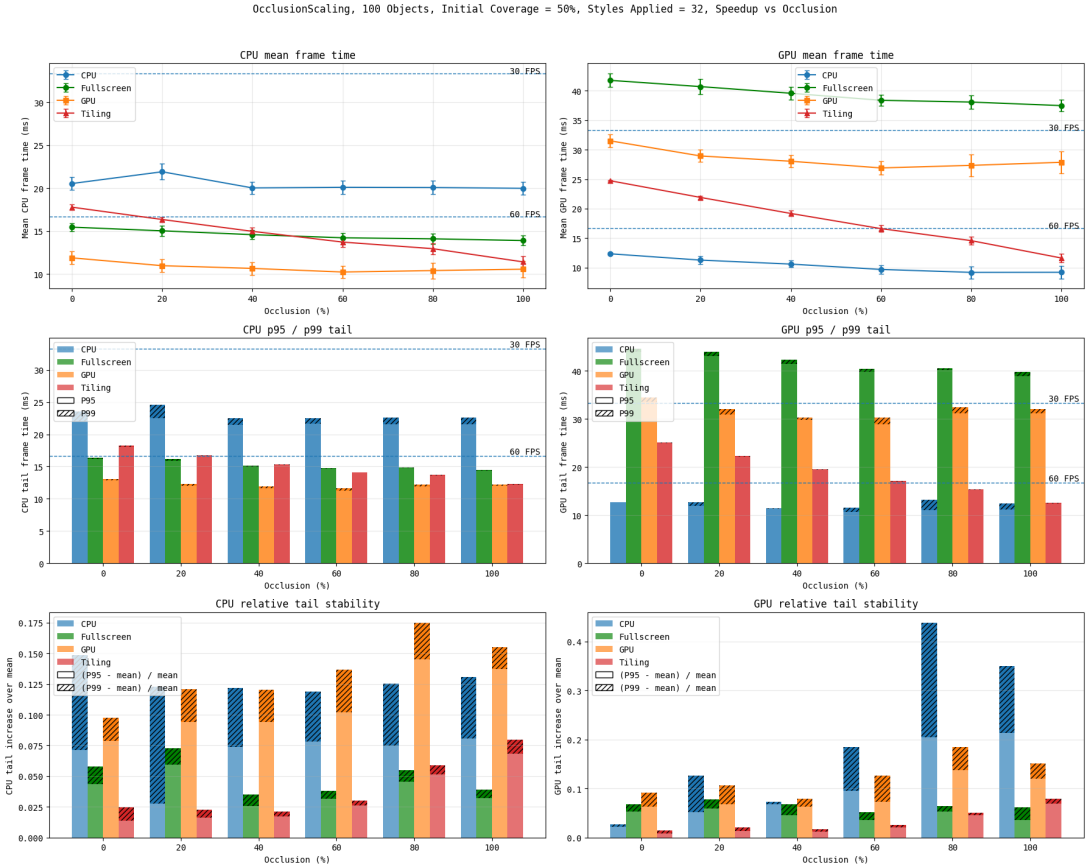


Figure D.7: Mean Frame Time, Tail and Stability Graphs for the Occlusion Scaling Test

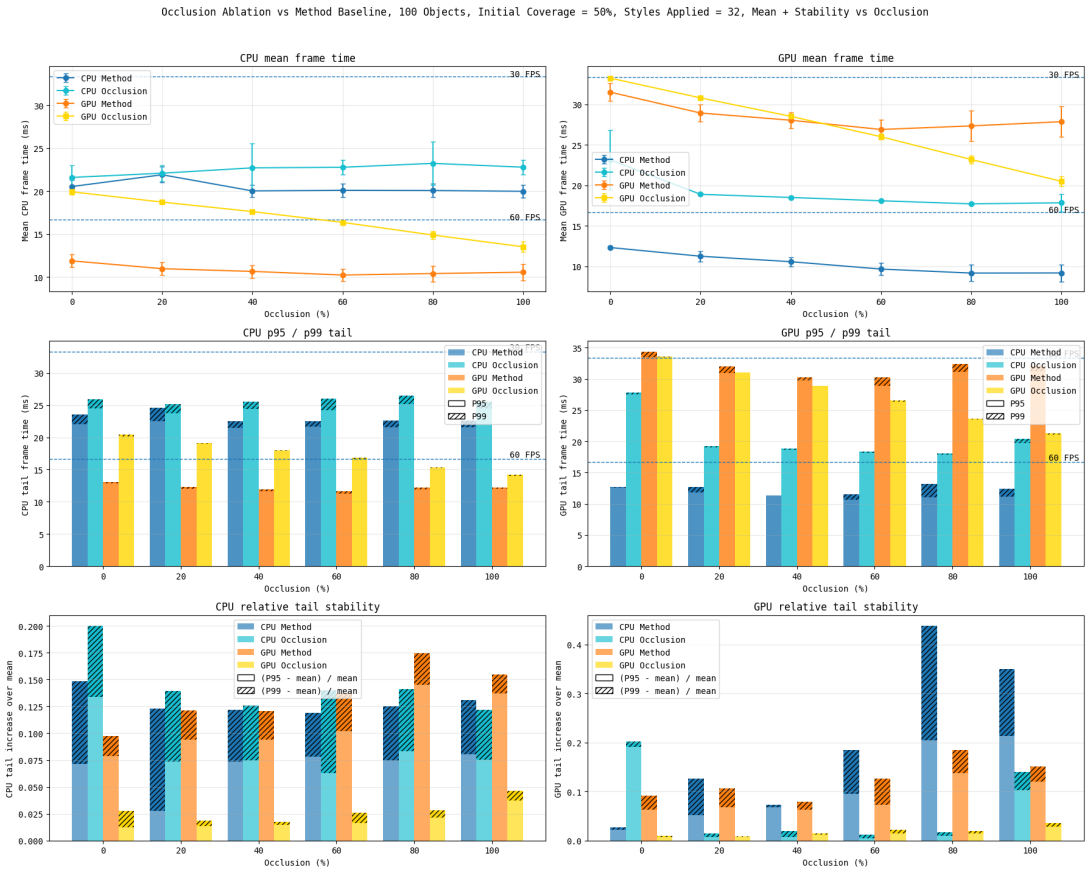


Figure D.8: Mean Frame Time, Tail and Stability Graphs for the Occlusion Ablation

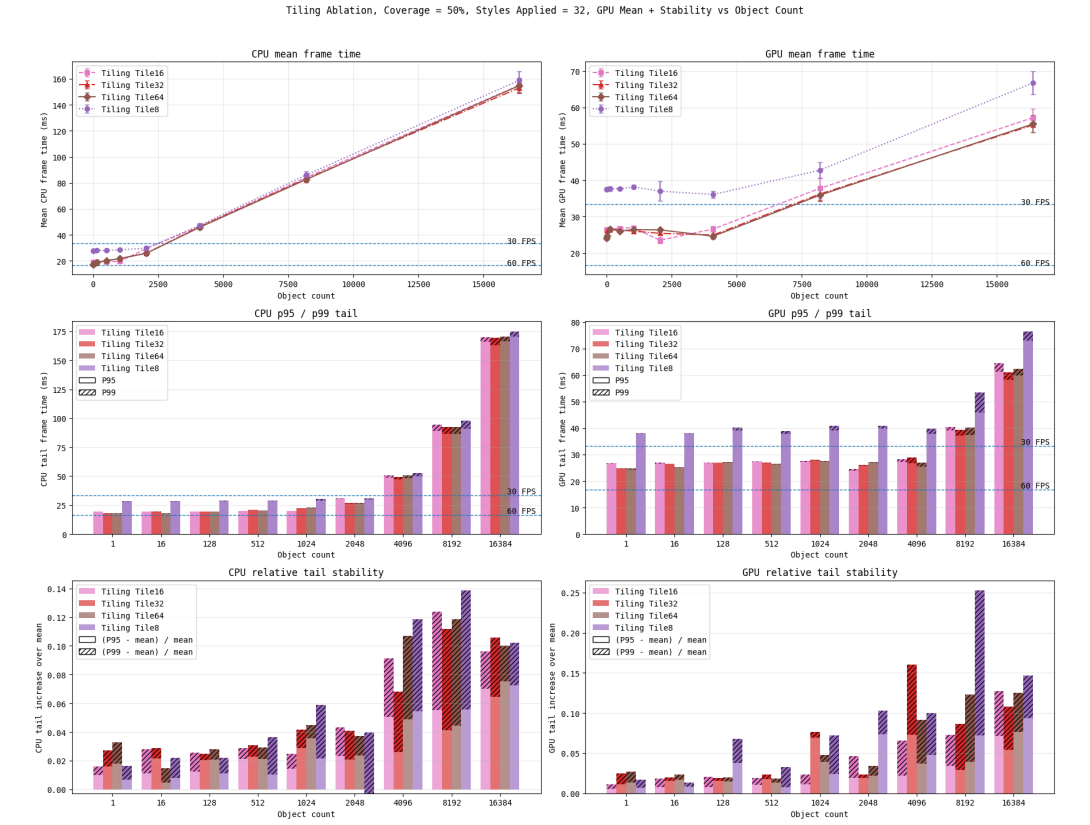


Figure D.9: Mean Frame Time, Tail and Stability Graphs for the Tiling Ablation on the Object Count Scaling Test

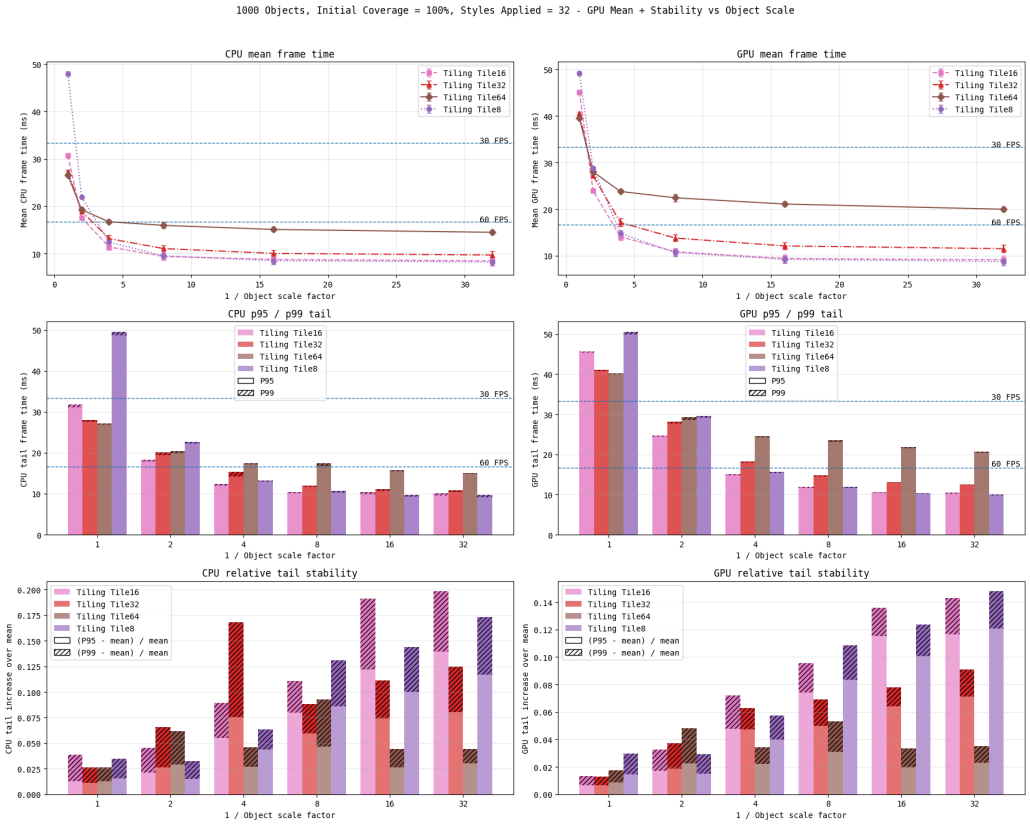


Figure D.10: Mean Frame Time, Tail and Stability Graphs for the Tiling Ablation on the Object Scale Test